

Computer Vision and Machine Learning for Scientific Visual Data Analytics

Lecture 2: Basics of CVML



Shu Kong
shu.kong@oist.jp



Roadmap

Teaching materials are available: <https://github.com/aimerykong/OIST-mini-course-CVML>

Lecture	Topics	Hands-on experience
1. Intro	CV/ML, deep learning, artificial intelligence	Configure programming environment using Google Drive and Colab; implement basic scripts in Python
2. Basics of CV/ML	training, validation, testing, features, metrics, nearest neighbor	Implement a machine learning pipeline for image classification
3. Classification	Logistic Regression, gradient descent, SVM	Train a linear classifier for pollen species recognition
4. Deep Learning	Multi-Layer Perceptron, feature learning, backpropagation, neural networks	Train a convolutional neural network (CNN) for pollen species recognition
5. Segmentation and Detection	semantic and instance segmentation, U-Net,	Train a neural network for pollen grain detection/segmentation
6. Foundation Models	CLIP , Segment Anything Model , DINOv2 , GroundingDINO , finetuning	Finetune a pretrained model for image classification and segmentation
7. Advanced Topics	open-set recognition, novel species recognition, phylogentic reconstruction via clustering	Implement algorithms to recognize unknown/anomalous/novel observations
8. Brainstorm & Talks	Volunteer to present	n/a

Outline

- **Data and labels**
- Features
- Training, validation, and test sets
- Metrics
- Machine-learning pipeline
- Example: Nearest Neighbor

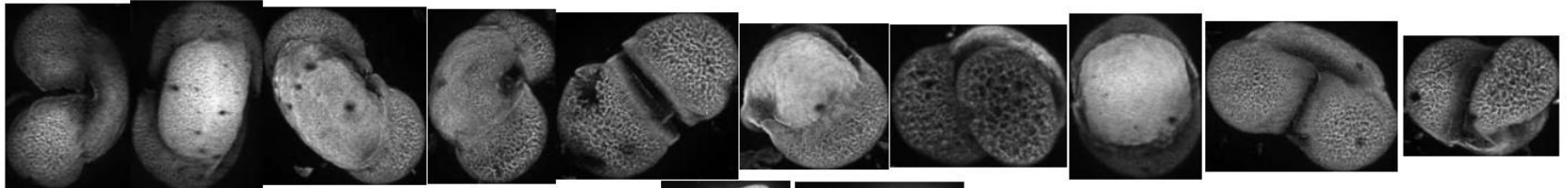
Data and labels

- Your dataset

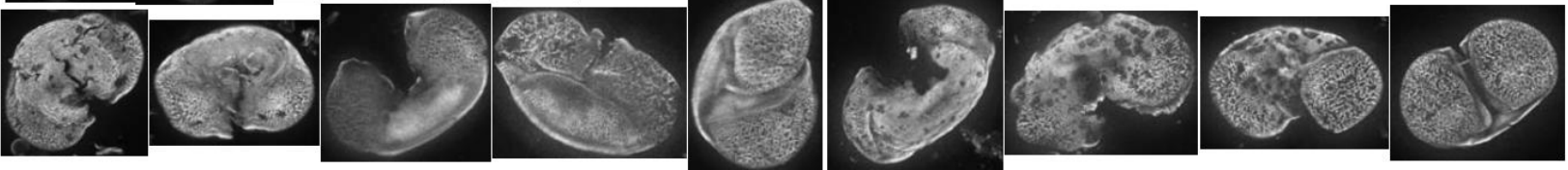
a training set

- data x
- targets y

P. critchfieldii



P. glauca



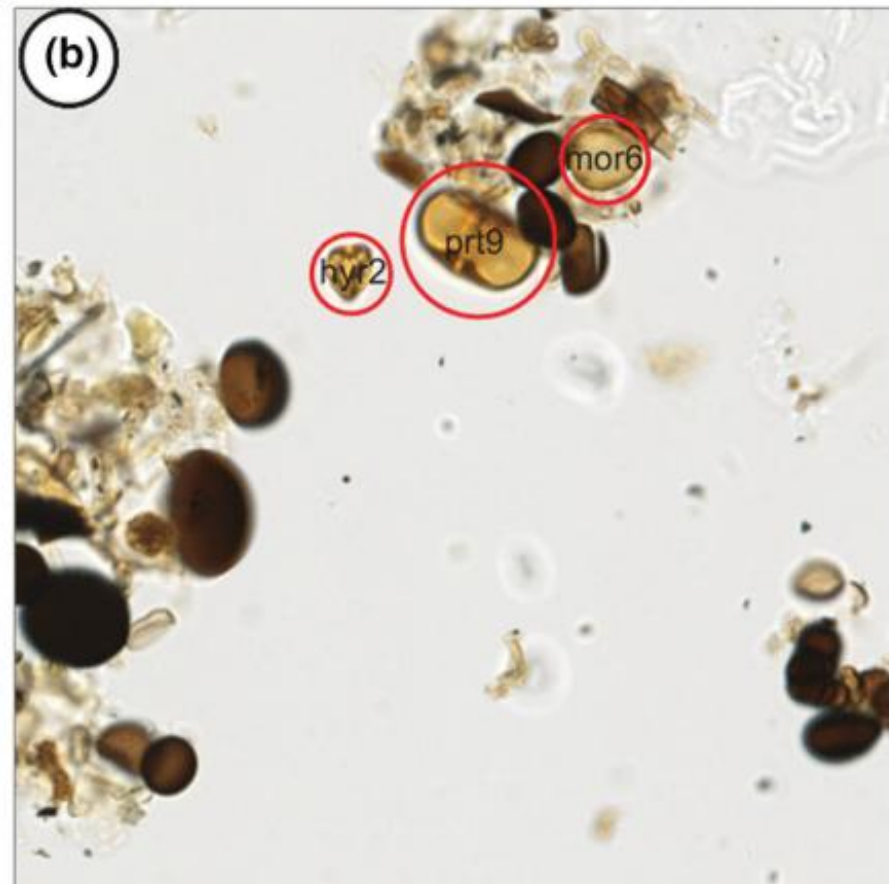
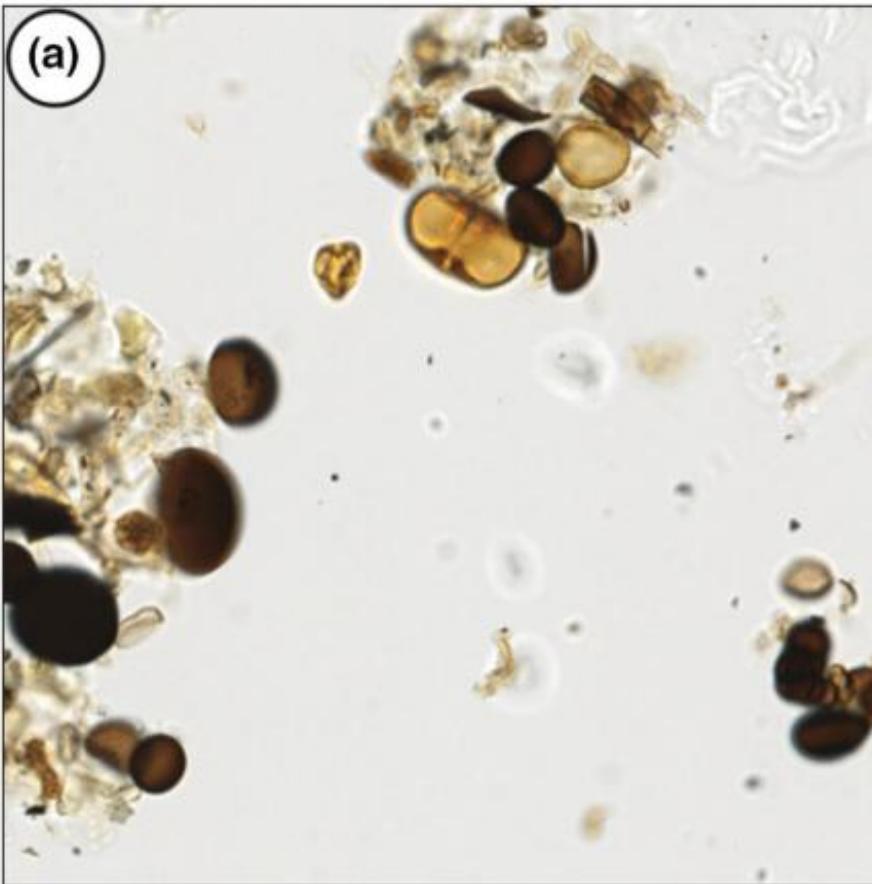
Data and labels

- Your dataset

a training set

- data x
- targets y

Pollen grains with species labels



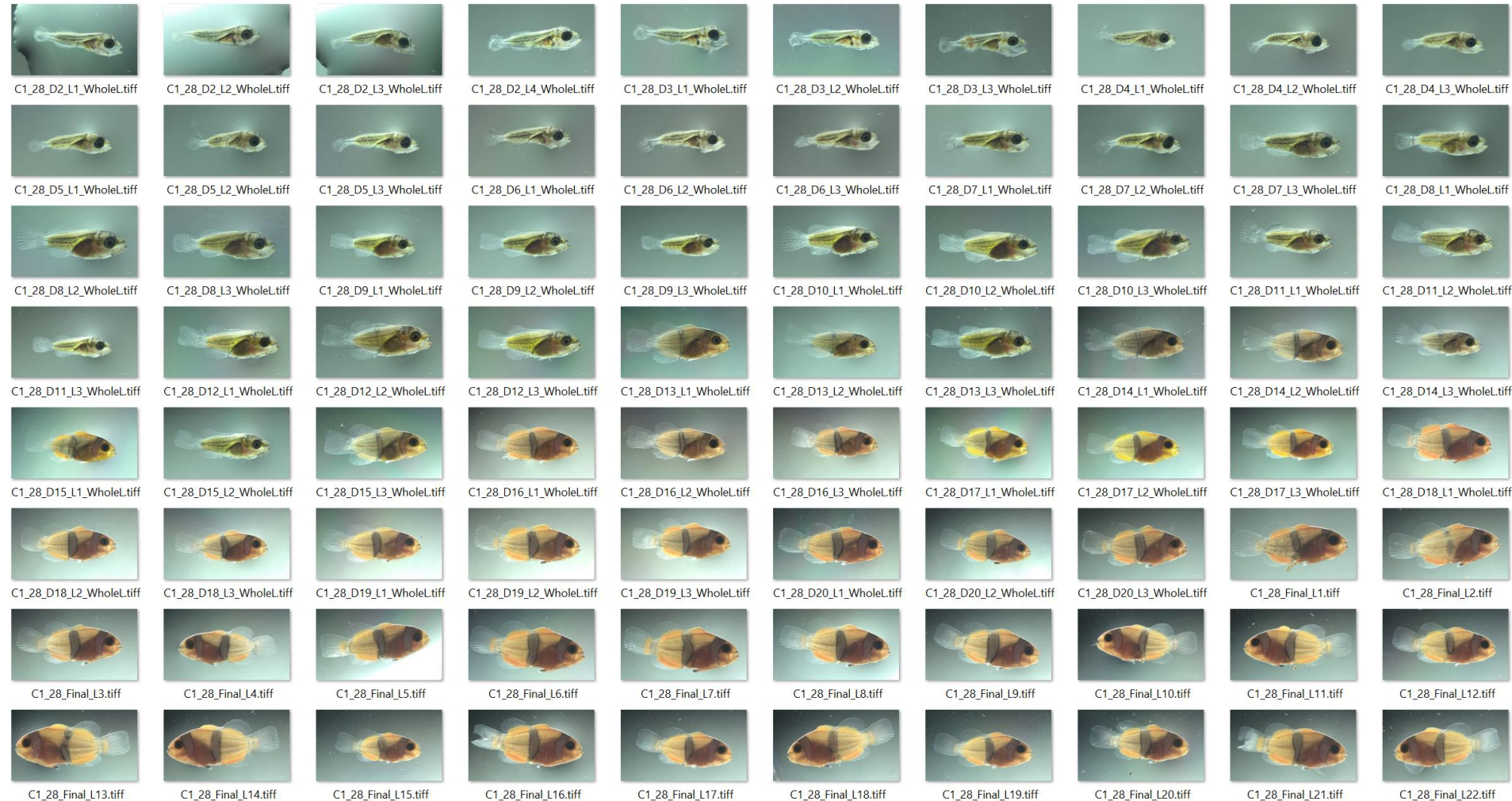
Data and labels

a training set

- data x
- targets y

- Your dataset

Images of fish labeled with specimen index, temperature, day after hatching, etc.



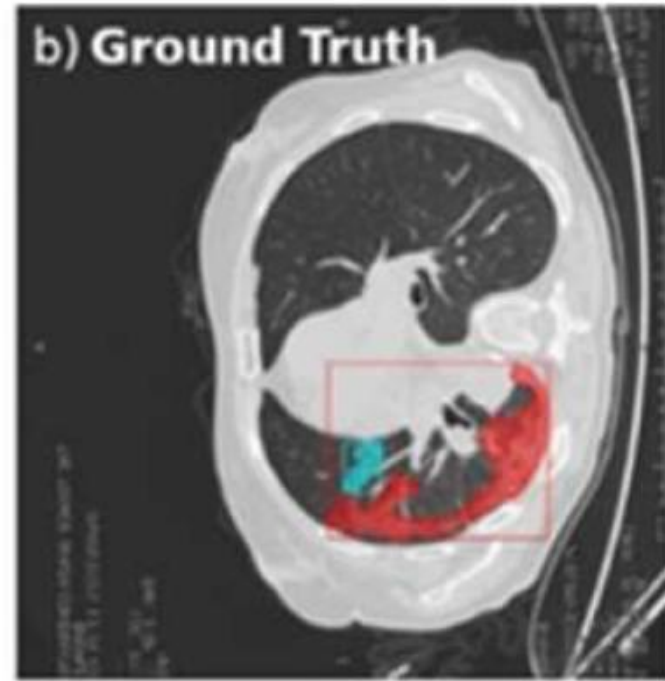
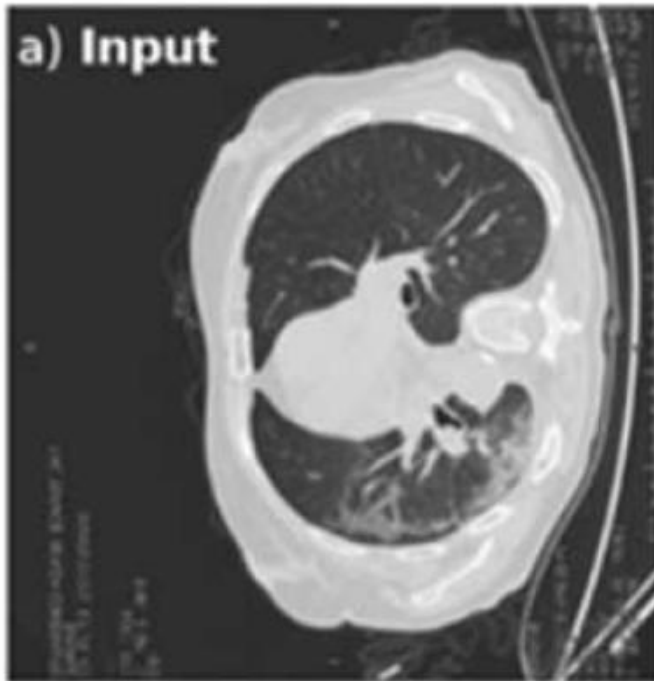
Data and labels

- Your dataset

a training set

- data x
- targets y

Lung infections



Outline

- Data and labels
- **Features**
- Metrics
- Training, validation, and test sets
- Machine-learning pipeline
- Example: Nearest Neighbor

Why using features to represent data?

1. makes processing computationally manageable
2. reduces the "noise" of unnecessary backgrounds
3. extracts the core shapes, textures, or colors required for accurate predictions.

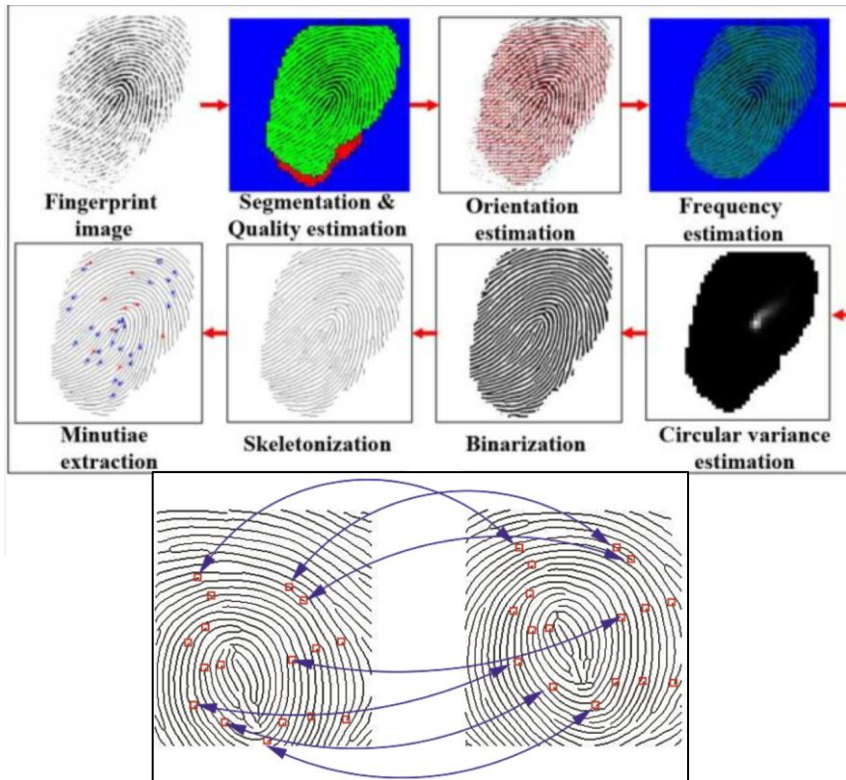
Features were handcrafted but are now learned via deep learning



Why using features to represent data?

1. makes processing computationally manageable
2. reduces the "noise" of unnecessary backgrounds
3. extracts the core shapes, textures, or colors required for accurate predictions.

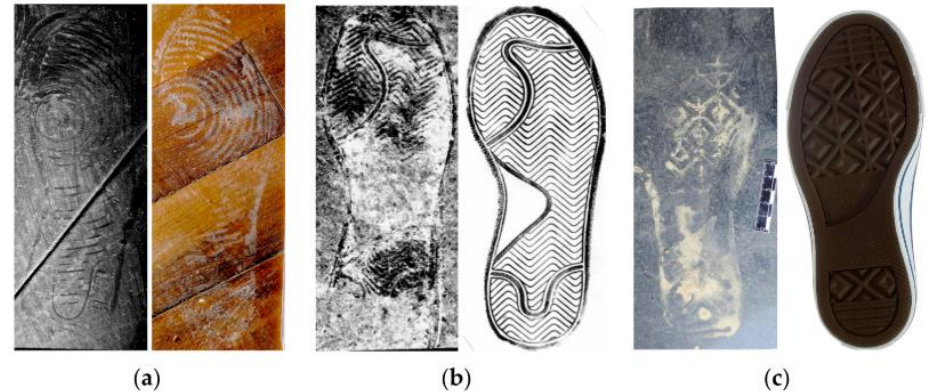
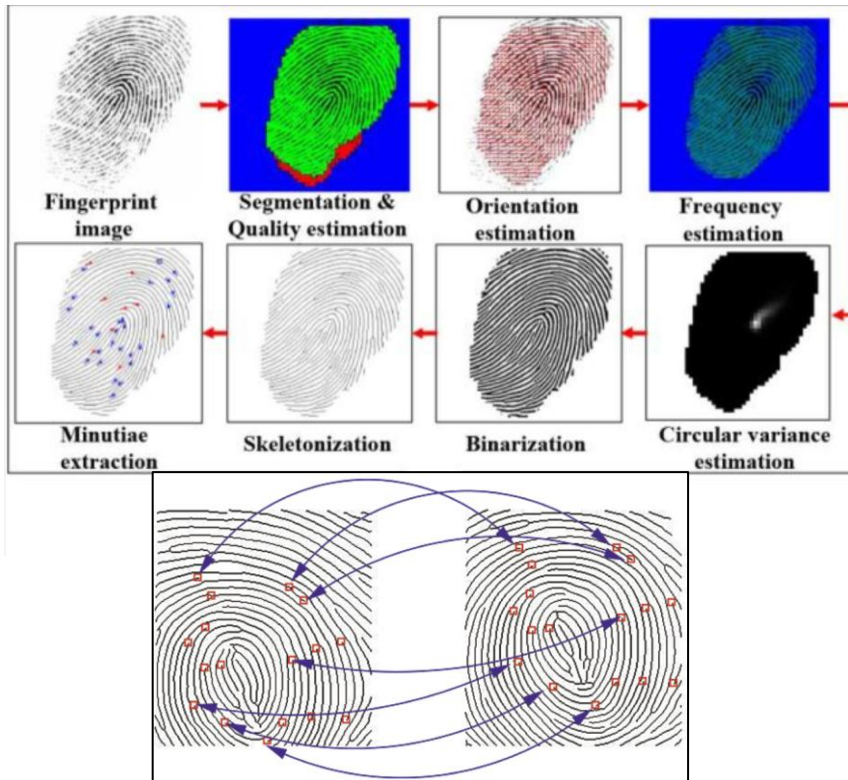
Features were handcrafted but are now learned via deep learning



Why using features to represent data?

1. makes processing computationally manageable
2. reduces the "noise" of unnecessary backgrounds
3. extracts the core shapes, textures, or colors required for accurate predictions.

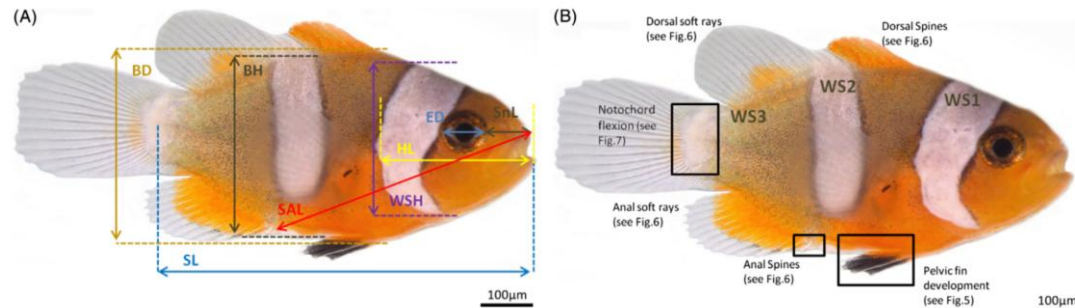
Features were handcrafted but are now learned via deep learning



Why using features to represent data?

1. makes processing computationally manageable
2. reduces the "noise" of unnecessary backgrounds
3. extracts the core shapes, textures, or colors required for accurate predictions.

Features were handcrafted but are now learned via deep learning



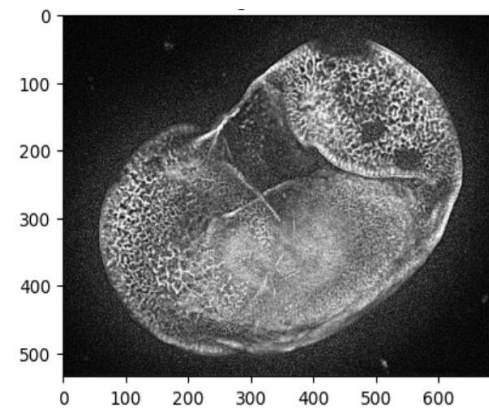
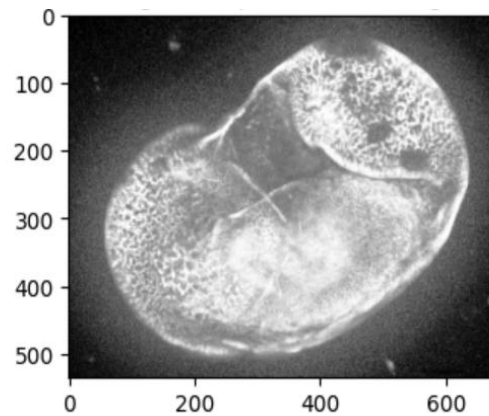
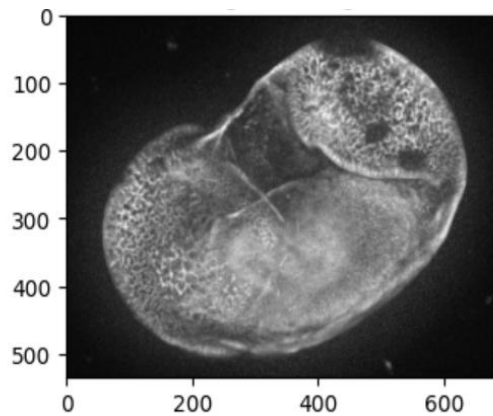
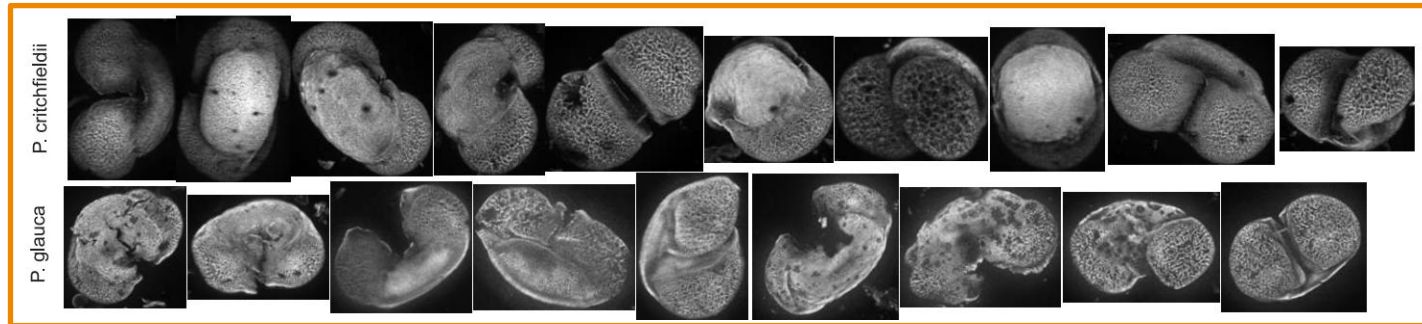
	Quantitative trait	Trait description
Quantitative traits	Standard length (SL)	Distance from the snout to the extremity of the notochord in preflexion larvae, and distance from the snout to the middle of the caudal peduncle in post flexion larvae
	Snout anus length (SAL)	Distance from the snout to the anus
	Head length (HL)	Distance from the snout to the extremity of the branchial opercular
	Body depth (BD)	Distance perpendicular to HL directly from the top of the head at the basis of the dorsal fin to the bottom of the body at the implantation of the pelvic fin
	Eye diameter (ED)	Distance at the longest axis of the eye
	Snout length (SL)	Distance from the snout to the first border of the eye
	Head Depth (HD)	Distance perpendicular to HL from the anus to the basis of the dorsal fin soft rays.
Qualitative traits	Anal soft rays	Appearance of the soft rays
	Anal spines	Appearance of the spines
	Dorsal soft rays	Appearance of the soft rays
	Dorsal spines	Appearance of the spines
	Pelvic fin	Development of the pelvic fin
	White stripes	Appearance of the three body white stripes
	Notochord flexion	Flexion steps of the notochord

[Roux, et al., 2019]

Data & features and labels

a training set

- data x
- targets y

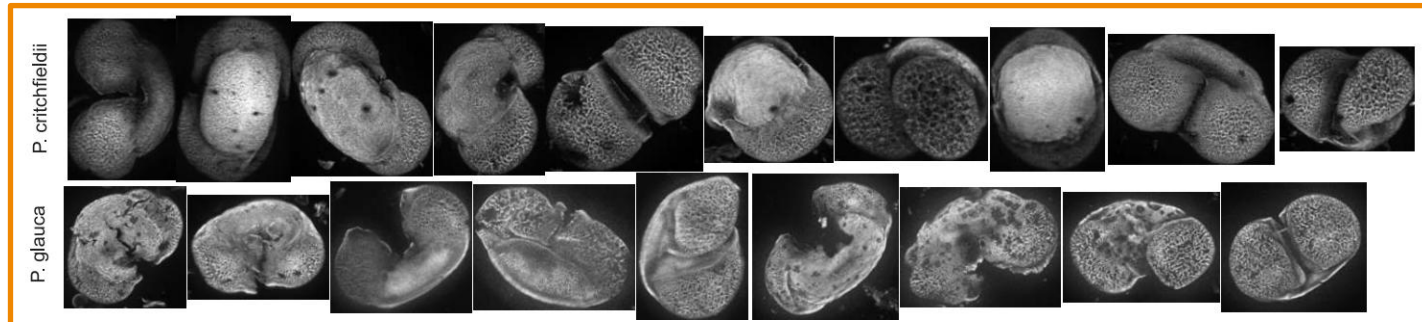


Raw pixels as features? Sure!
Enhanced pixels as features? Fine!
Manually designed features? Good!
Auto-learned features? Great!

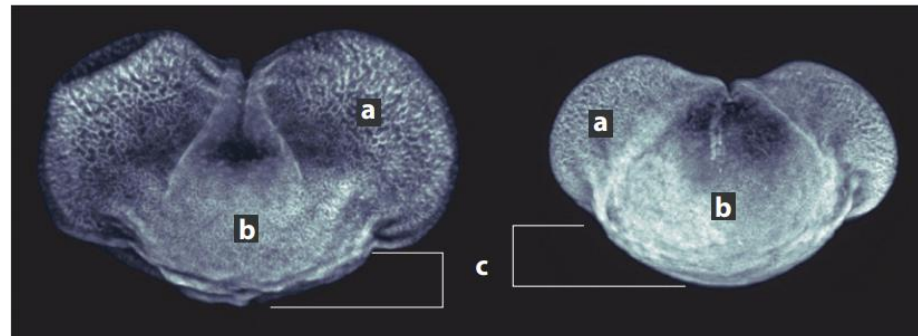
Data&features and labels

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



feature x :



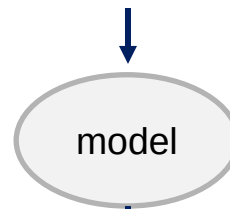
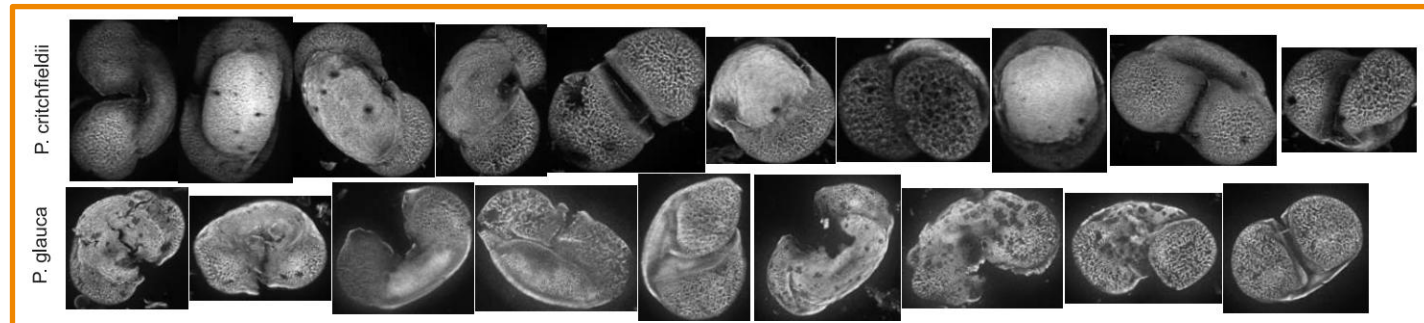
size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

Raw pixels as features? Sure!
Enhanced pixels as features? Fine!
Manually designed features? Good!
Auto-learned features? Great!

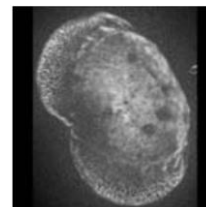
Data & features and labels

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



Train a model (i.e., teach a machine) to recognize the species of a test image x'



x'
class-1 or class-2?

feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

Raw pixels as features? Sure!
Enhanced pixels as features? Fine!
Manually designed features? Good!
Auto-learned features? Great!

Outline

- Data and labels
- Features
- **Metrics**
- Training, validation, and test sets
- Machine-learning pipeline
- Example: Nearest Neighbor

How to tell if one model is better than the other one?



How to tell if one model is better than the other one?

1. Using quantifiable measures to assess the performance of a model
2. Metrics provide a concrete basis to validate outcomes and compare models.
3. The right metric depends on your specific task.



How to tell if one model is better than the other one?

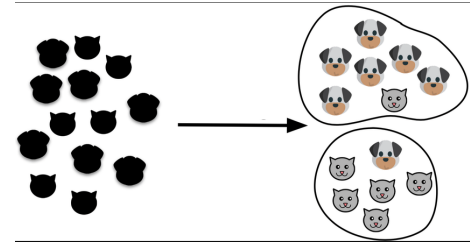
1. Using quantifiable measures to assess the performance of a model
2. Metrics provide a concrete basis to validate outcomes and compare models.
3. The right metric depends on your specific task.

Model-A is better than Model-B.... in terms of what?

- Larger/smaller in parameter size: larger capacity/capability vs. lightweight for mobile?
- Faster/slower in runtime, on GPU vs. CPU?
- More accurate in classifying data (on your dataset)?
- Higher recall in detecting cancer cells?
- Cheaper to deploy in a laptop?
- Highly specialized vs. more generalizable?



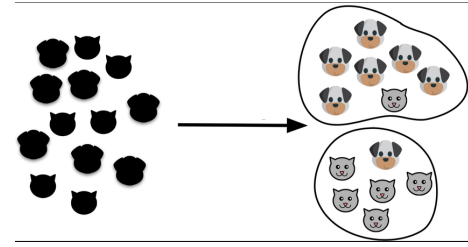
How to tell if one model is better than the other one?



1. Using quantifiable measures to assess the performance of a model
2. Metrics provide a concrete basis to validate outcomes and compare models.
3. The right metric depends on your specific task.

e.g., for binary classification, we consider correct predictions and incorrect ones

Examples of metrics



1. Using quantifiable measures to assess the performance of a model
2. Metrics provide a concrete basis to validate outcomes and compare models.
3. The right metric depends on your specific task.

e.g., for binary classification, we consider correct predictions and incorrect ones

		ACTUAL	
		Positive	Negative
PREDICTED	Positive	True Positive (TP) 5	False Positive (FP) 10
	Negative	False Negative (FN) 15	True Negative (TN) 70

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = 0.75$$

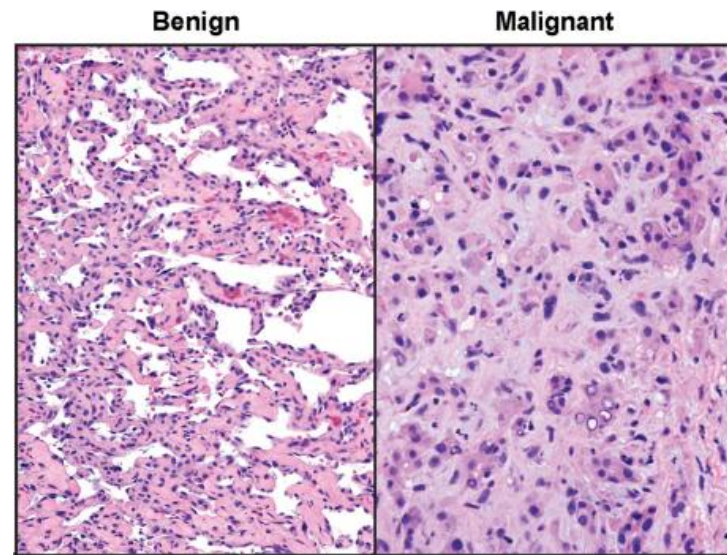
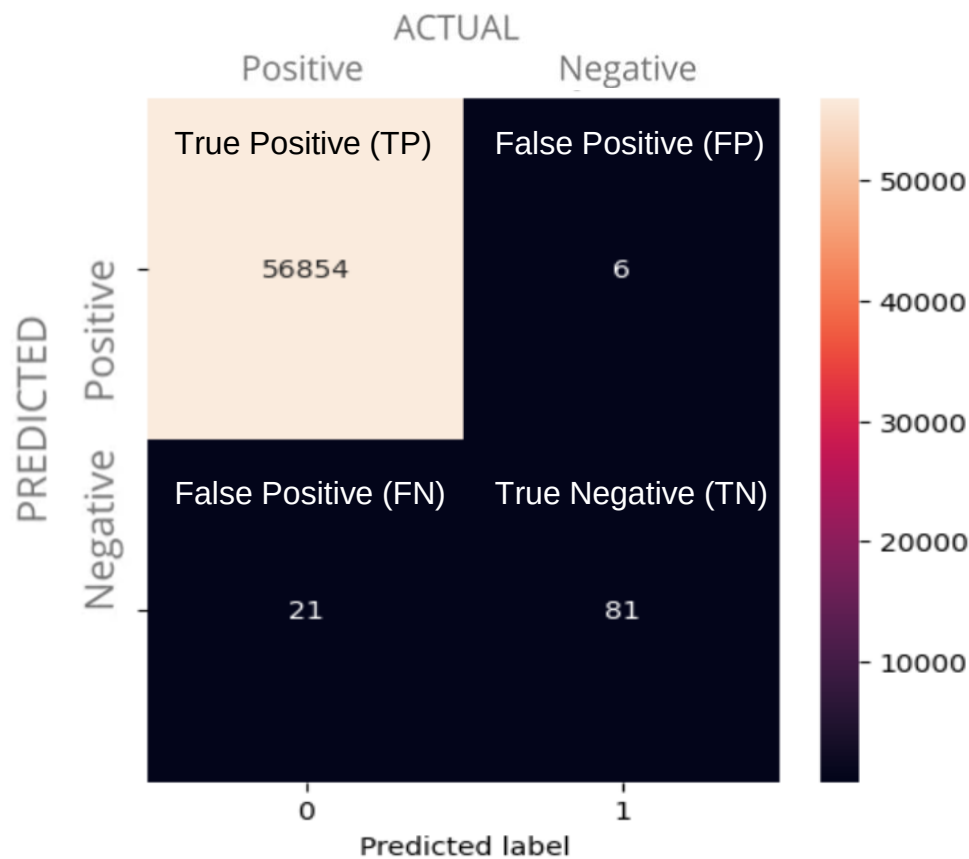
$$\text{Recall} = \frac{TP}{TP + FN} = 0.25$$

$$\text{Precision} = \frac{TP}{TP + FP} = 0.33$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.28$$

Caution on metrics choices!

Imbalanced data: rare observations



$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = 0.9995$$

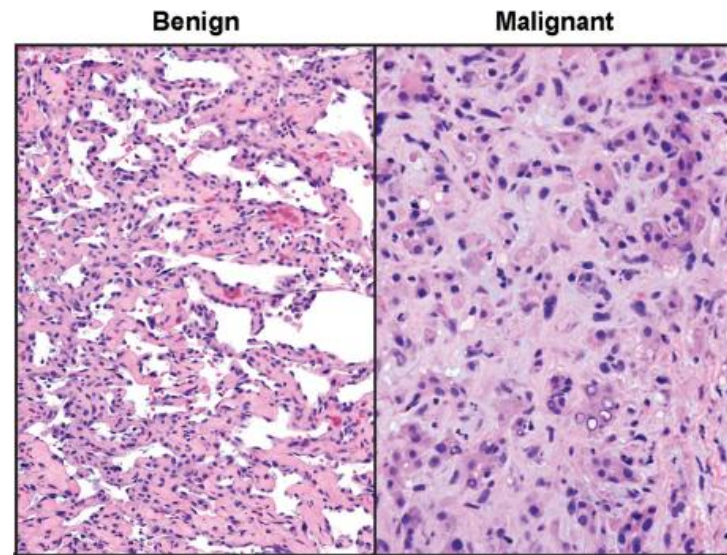
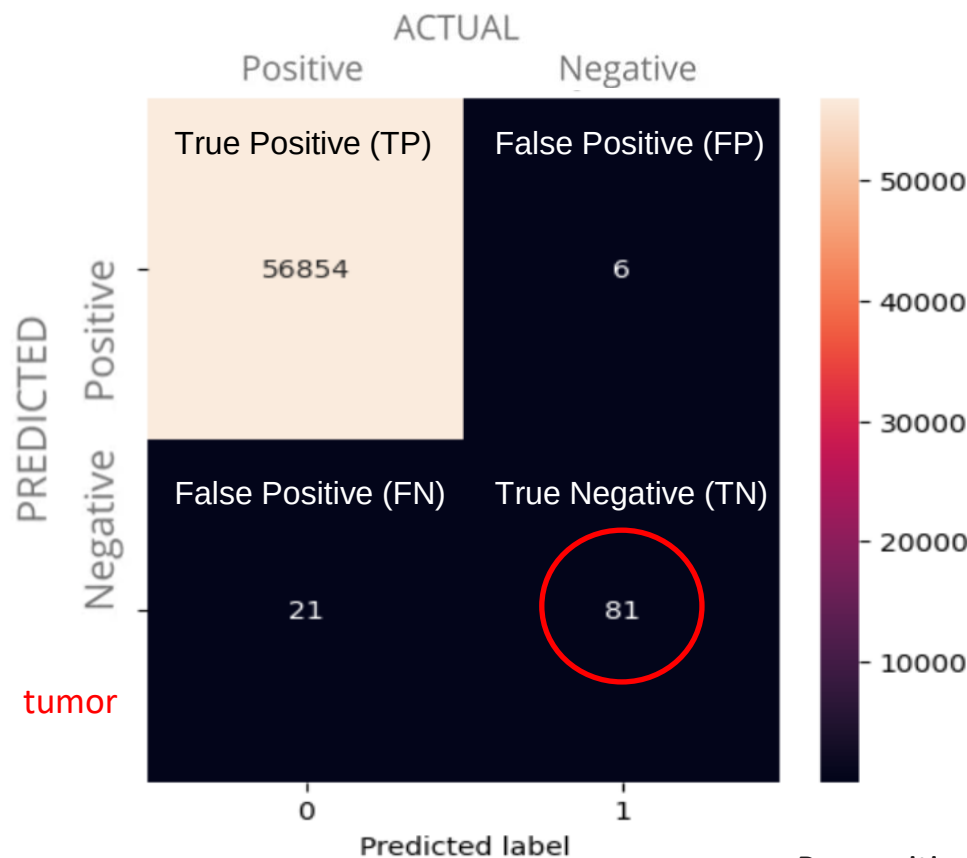
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = 0.9996$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = 0.9999$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.9997$$

Caution on metrics choices!

Imbalanced data: rare observations



$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = 0.9995$$

$$\text{Recall} = \frac{TP}{TP + FN} = 0.9996$$

$$\text{Precision} = \frac{TP}{TP + FP} = 0.9999$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.9997$$

Recognition accuracy of tumor

- Negative predictive value: $81 / (81 + 21) = 0.7941$

Various choices!

Confusion matrix

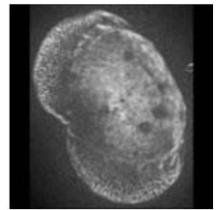
		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Total population = P + N		
	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

https://en.wikipedia.org/wiki/Confusion_matrix

		Predicted condition		Sources: [13][14][15][16][17][18][19][20] view · talk · edit		
		Total population = P + N	Predicted positive	Predicted negative	Informedness, bookmaker informedness (BM) = TPR + TNR − 1	Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$
Actual condition	Real Positive (P) ^[a]	True positive (TP), hit ^[b]	False negative (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{\text{P}} = 1 - \text{FNR}$	False negative rate (FNR), miss rate type II error ^[c] $= \frac{\text{FN}}{\text{P}} = 1 - \text{TPR}$	
	Real Negative (N) ^[d]	False positive (FP), false alarm, overestimation	True negative (TN), correct rejection ^[e]	False positive rate (FPR), probability of false alarm, fall-out type I error ^[f] $= \frac{\text{FP}}{\text{N}} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{\text{N}} = 1 - \text{FPR}$	
Prevalence $= \frac{\text{P}}{\text{P} + \text{N}}$		Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{TP} + \text{FP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{TN} + \text{FN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR−) $= \frac{\text{FNR}}{\text{TNR}}$	
Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$		False discovery rate (FDR) $= \frac{\text{FP}}{\text{TP} + \text{FP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{TN} + \text{FN}} = 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) $= \text{PPV} + \text{NPV} - 1$	Diagnostic odds ratio (DOR) $= \frac{\text{LR}+}{\text{LR}-} = \frac{\text{TP} \times \text{TN}}{\text{FP} \times \text{FN}}$	
Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$		F ₁ score $= \frac{2 \text{ PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}}$ $= \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}}$	Fowlkes– Malloves index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	phi or Matthews correlation coefficient (MCC) $= \frac{\sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}}}{\sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$	

Outline

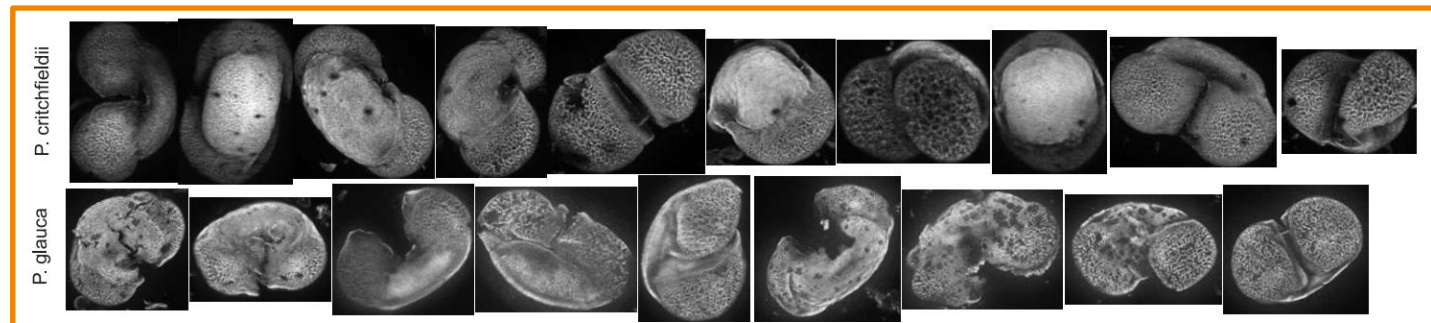
- Data and labels
- Features
- Metrics
- **Training, validation, and test sets**
- Machine-learning pipeline
- Example: Nearest Neighbor

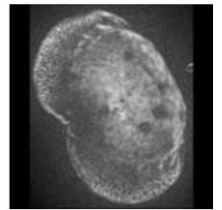


Training, validation and test data

Data

- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?

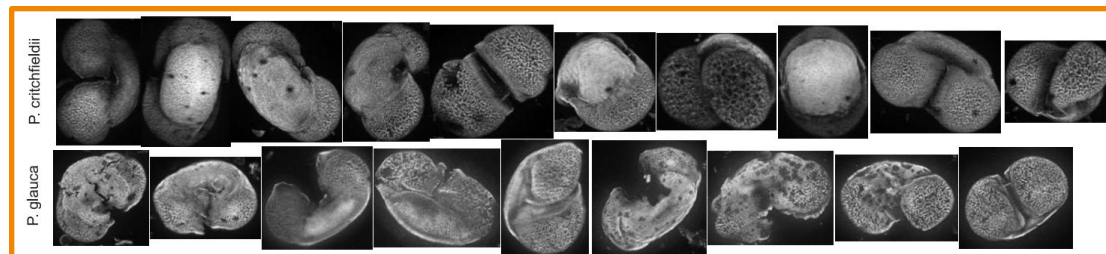
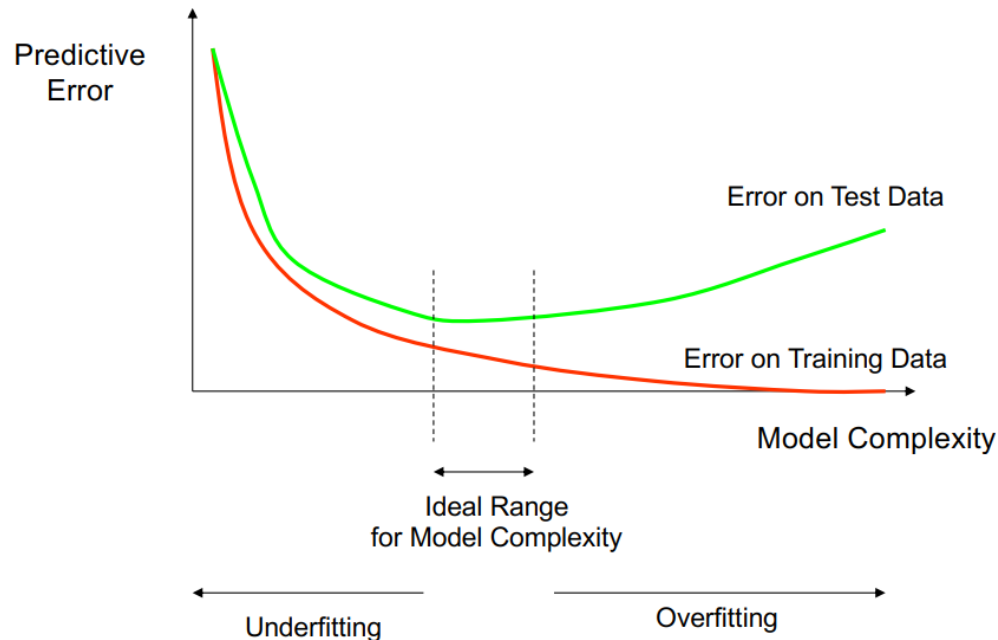


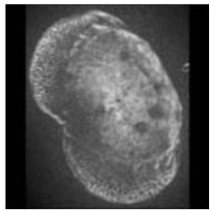


Training, validation and test data

Data

- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?





Training, validation and test data

Data

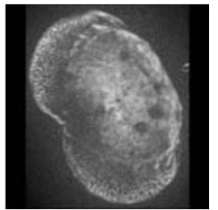
- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?

Approach 1: Split into train and validation data

Training Data

validation data

- Learn parameters of each model from training data
- Evaluate all models on validation data, and pick the best performer



Training, validation and test data

Data

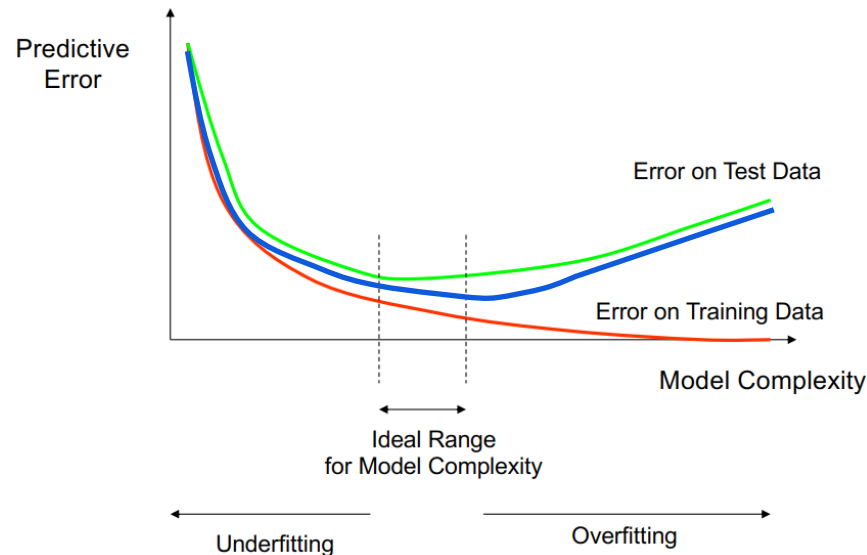
- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?

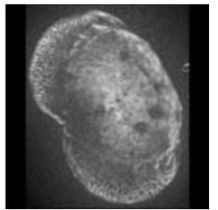
Approach 1: Split into train and validation data

Training Data

validation data

- Learn parameters of each model from training data
- Evaluate all models on validation data, and pick the best performer





Training, validation and test data

Data

- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?

Approach 1: Split into train and validation data

Training Data

validation data

- Learn parameters of each model from training data
- Evaluate all models on validation data, and pick the best performer

More training data & less validation data

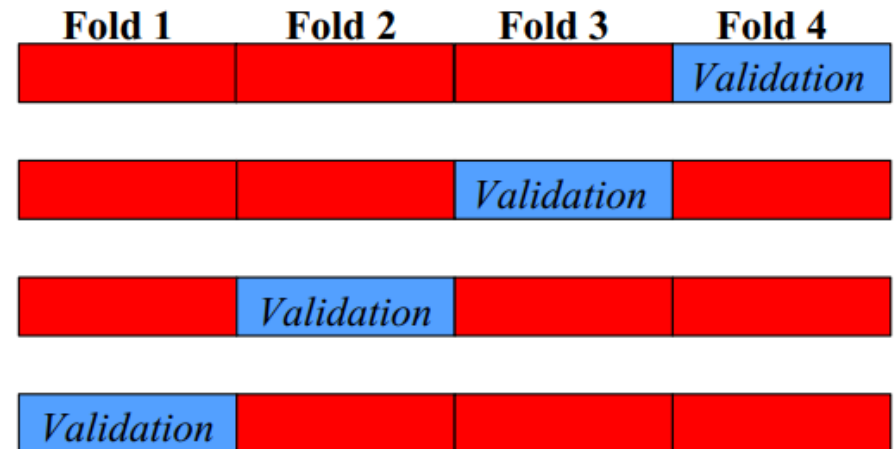
vs.

less training data & more validation data

Training, validation and test data

Data

- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
 - How can we decide which is best?
-
- Divide training data into K equal-sized *olds*
 - Train on K-1 folds, evaluate on remainder
 - Pick model with best average performance across K trials

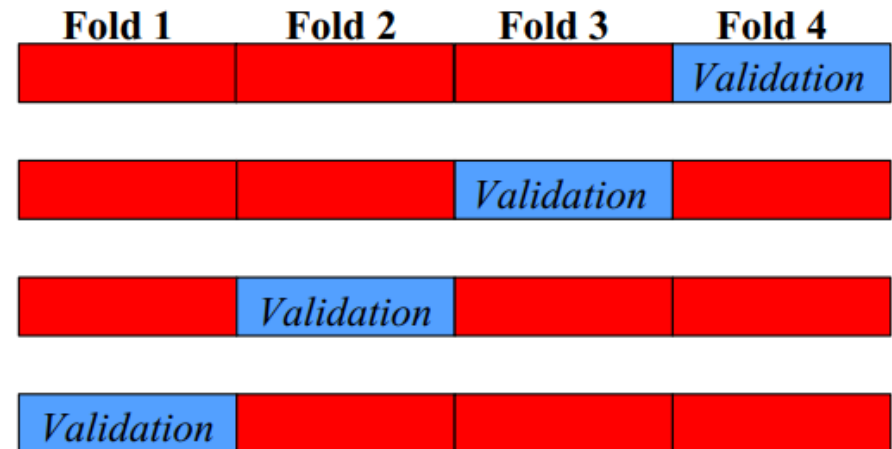


Training, validation and test data

Data

- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?

- Divide training data into K equal-sized *folds*
- Train on K-1 folds, evaluate on remainder
- Pick model with best average performance across K trials

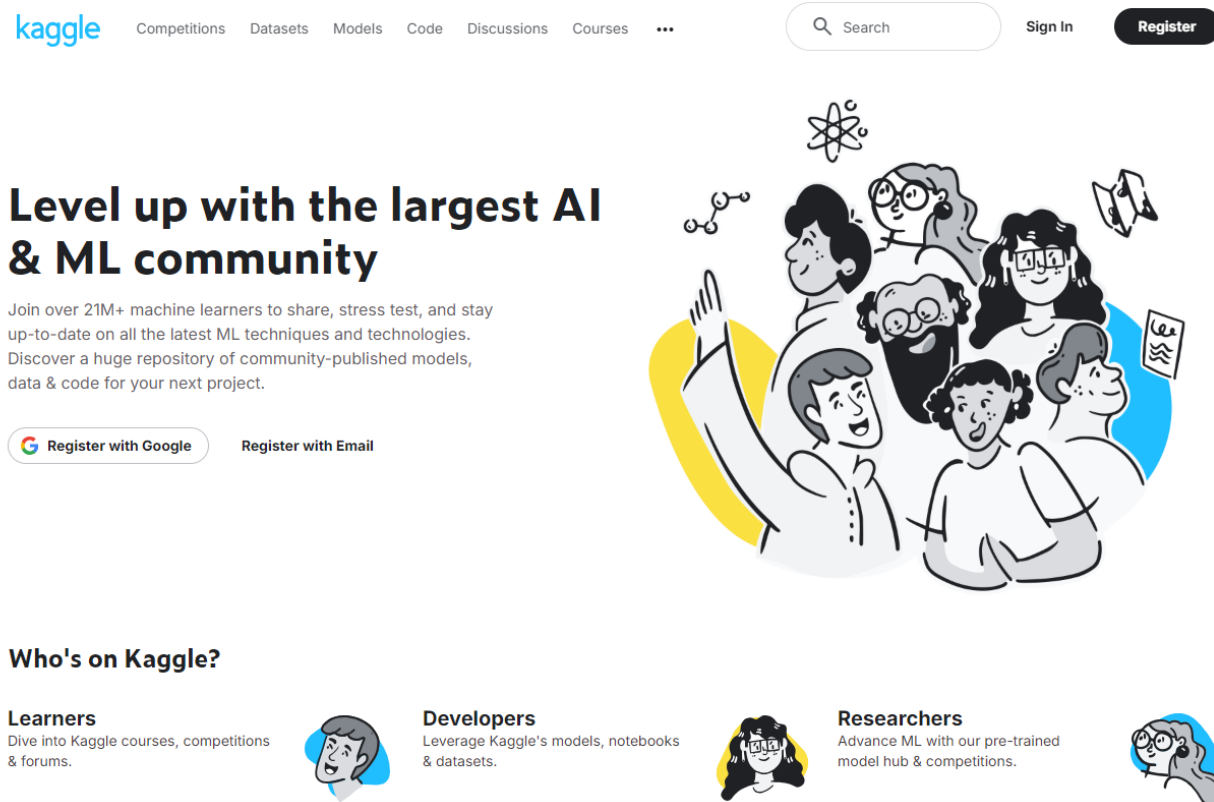


How many folds?

- *Bias*: Too few, and effective training dataset much smaller
- *Variance*: Too many, and test performance estimates noisy
- *Cost*: Must run training algorithm once per fold (parallelizable)
- *Practical rule of thumb*: 5-fold or 10-fold cross-validation
- *Theoretically troubled*: Leave-one-out cross-validation, $K=N$

Competitions

- Training data
 - Used to build your model(s)
- Validation data
 - Used to assess, select among, or combine models
 - Personal validation; leaderboard; ...
- Test data
 - Used to estimate “real world” performance



The image shows the Kaggle website's header and hero section. The header includes the Kaggle logo, navigation links for Competitions, Datasets, Models, Code, Discussions, and Courses, a search bar, and buttons for Sign In and Register. The hero section features a large illustration of a diverse group of people, some with icons above their heads representing different fields like science (atom), chemistry (molecule), and music (note). Below the illustration, the text reads 'Level up with the largest AI & ML community' followed by a description of the community's size and resources. At the bottom, there are two registration buttons: 'Register with Google' and 'Register with Email'. Below the hero section, there is a section titled 'Who's on Kaggle?' with three categories: Learners, Developers, and Researchers, each with a brief description and a small icon.

kaggle Competitions Datasets Models Code Discussions Courses ... Search Sign In Register

Level up with the largest AI & ML community

Join over 21M+ machine learners to share, stress test, and stay up-to-date on all the latest ML techniques and technologies. Discover a huge repository of community-published models, data & code for your next project.

Register with Google Register with Email

Who's on Kaggle?

Learners

Dive into Kaggle courses, competitions & forums.



Developers

Leverage Kaggle's models, notebooks & datasets.



Researchers

Advance ML with our pre-trained model hub & competitions.



Competitions

- Training data
 - Used to build your model(s)
- Validation data
 - Used to assess, select among, or combine models
 - Personal validation; leaderboard; ...
- Test data
 - Used to estimate “real world” performance

<https://autoexpert-arena.github.io/>

```
ASA2026_dataset/  
|-- insecta/  
|   |-- train/      # training images  
|   |-- test/       # test images  
|   |-- ...  
|-- weeds/  
|   |-- train/  
|   |-- test/  
|   |-- ...  
|-- mollusca/  
|   |-- train/  
|   |-- test/  
|   |-- ...
```



🏆 CVPR 2026 Auto-Annotation Challenge

Track on Automated Species Annotation for Ecological/Biological Research

Overview

👋 Welcome to the CVPR 2026 Auto-Annotation Challenge, organized under the [AutoExpert workshop](#).

Automated Species Annotation (ASA) is critical for biodiversity assessment, conservation, and ecological research. Training accurate species-level models typically requires large-scale annotated datasets, but expert annotation is costly and time-consuming, making it feasible to label only a few examples per species. This competition invites participants to develop models that can recognize species from expert-crafted annotation guidelines, which provide a **few images and descriptions** for each species. Please refer to our technical report for a deeper understanding of the task [Auto-Annotation of Species](#).

Top-ranked teams are invited to speak but must submit a technical report to show how to reproduce the results and and make the submission visible by the end of the competition. Invited teams are expected to beat our official baseline. **We require all teams to clearly describe any foundation models and data being used in their methods.**

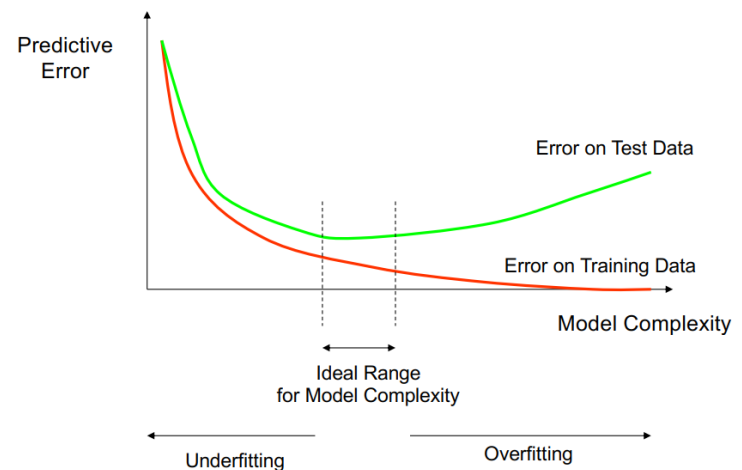
Good to know this!

- Assume training data are iid (independent & identically distributed samples) from some distribution $p(x,y)$
- Define “risk” and “empirical risk”
 - These are just “long term” test and observed training error

$$R(\theta) = \text{TestError} = \mathbb{E}[\mathbb{1}[c \neq \hat{c}(x; \theta)]]$$

$$R^{\text{emp}}(\theta) = \text{TrainError} = \frac{1}{m} \sum_i \mathbb{1}[c^{(i)} \neq \hat{c}(x^{(i)}; \theta)]$$

- How are these related? Depends on overfitting...



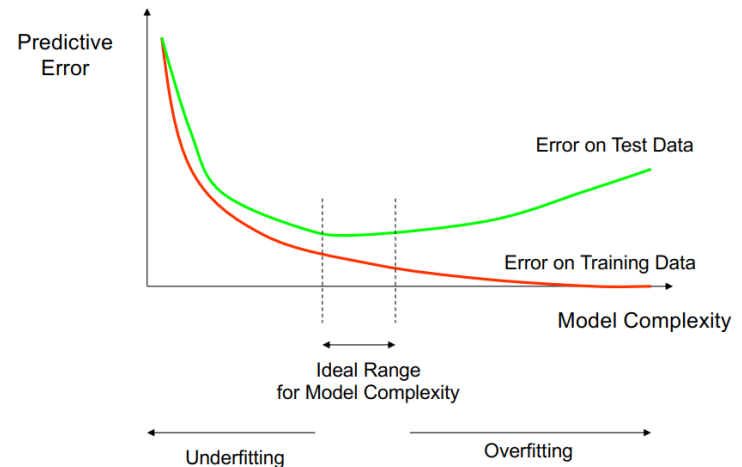
Good to know this!

- Assume training data are iid (independent & identically distributed samples) from some distribution $p(x,y)$
- Define “risk” and “empirical risk”
 - These are just “long term” test and observed training error

$$R(\theta) = \text{TestError} = \mathbb{E}[\mathbb{1}[c \neq \hat{c}(x; \theta)]]$$

$$R^{\text{emp}}(\theta) = \text{TrainError} = \frac{1}{m} \sum_i \mathbb{1}[c^{(i)} \neq \hat{c}(x^{(i)}; \theta)]$$

- How are these related? Depends on overfitting...
 - Underfitting domain: pretty similar...
 - Overfitting domain: test error might be lots worse!



Good to know this!

- Given some classifier, let H be its VC dimension
 - Represents “representational power” of classifier

$$R(\theta) = \text{TestError} = \mathbb{E}[\mathbb{1}[c \neq \hat{c}(x; \theta)]]$$

$$R^{\text{emp}}(\theta) = \text{TrainError} = \frac{1}{m} \sum_i \mathbb{1}[c^{(i)} \neq \hat{c}(x^{(i)}; \theta)]$$

- With “high probability” $(1-\eta)$, Vapnik showed

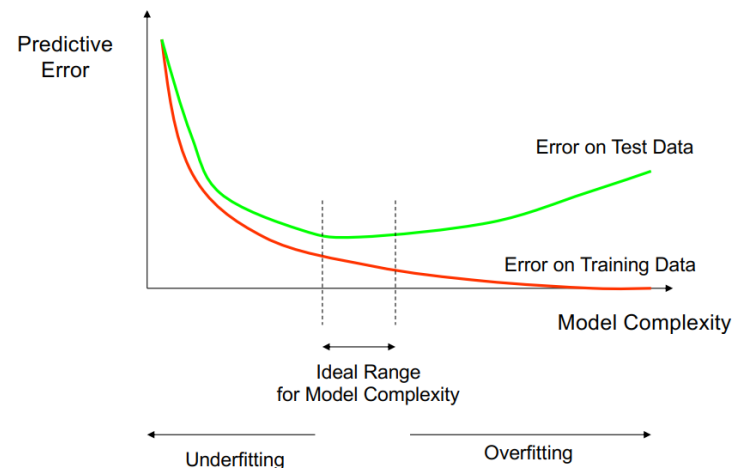
$$\text{TestError} \leq \text{TrainError} + \sqrt{\frac{H \log(2m/H) + H - \log(\eta/4)}{m}}$$



Vladimir Vapnik



Alexey Chervonenkis



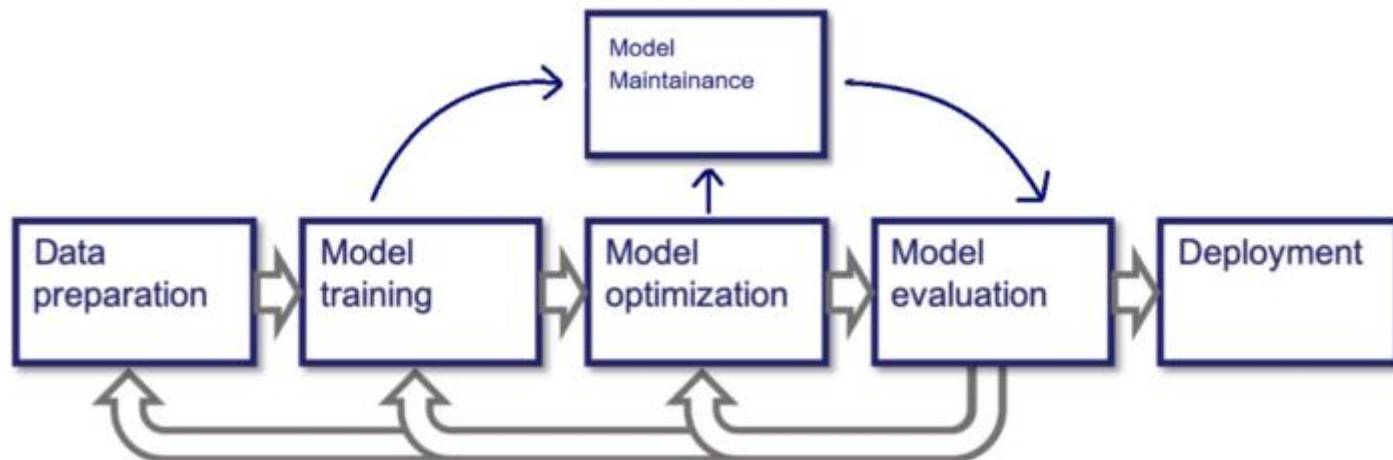
Outline

- Data and labels
- Features
- Metrics
- Training, validation, and test sets
- **Machine-learning pipeline**
- Example: Nearest Neighbor

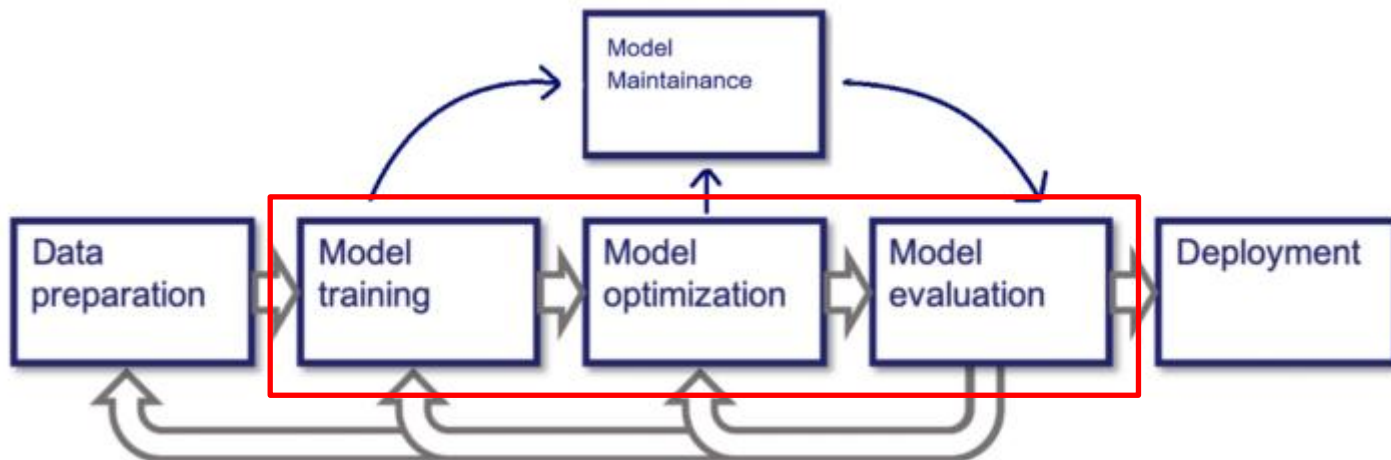
An Overall Pipeline of Machine Learning



An Overall Pipeline of Machine Learning

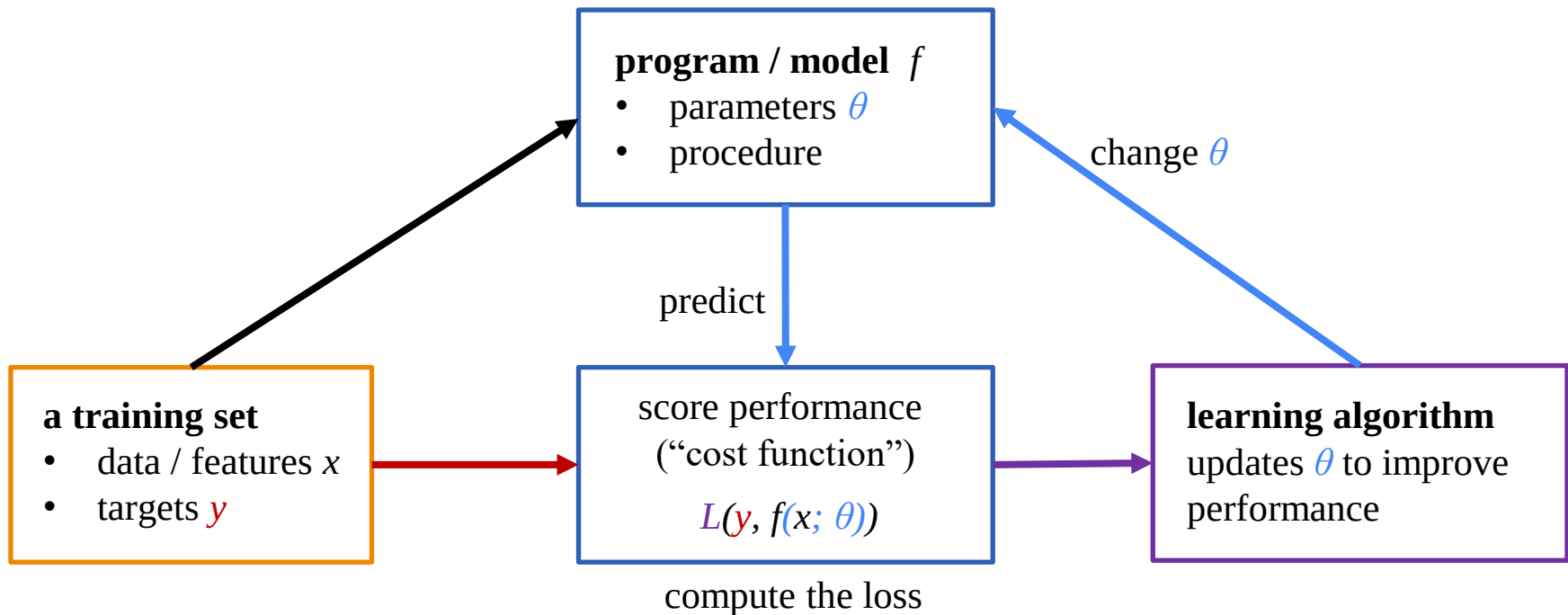


An Overall Pipeline of Machine Learning



An Overall Pipeline of Machine Learning

- Given a training set of labeled data
- Design a model with some parameters
- Formulate your task as a loss function (e.g., classification errors)
- Update the parameters to reduce the loss function over your training data

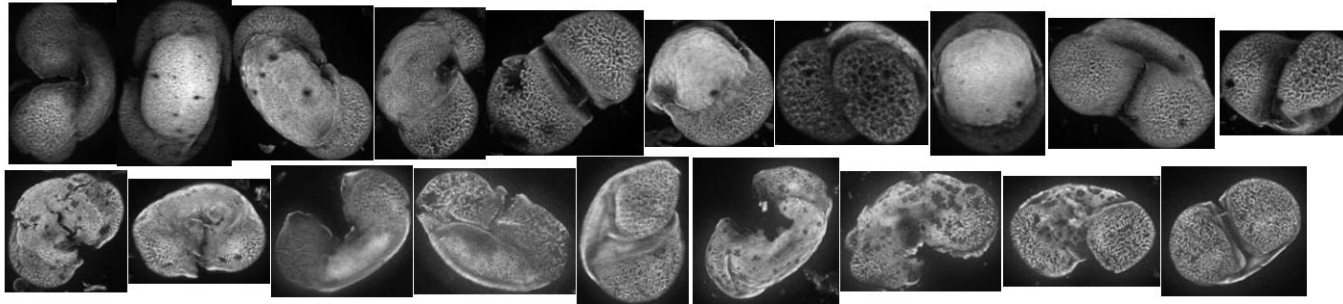


Outline

- Data and labels
- Features
- Metrics
- Training, validation, and test sets
- Machine-learning pipeline
- **Example: Nearest Neighbor**

Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images

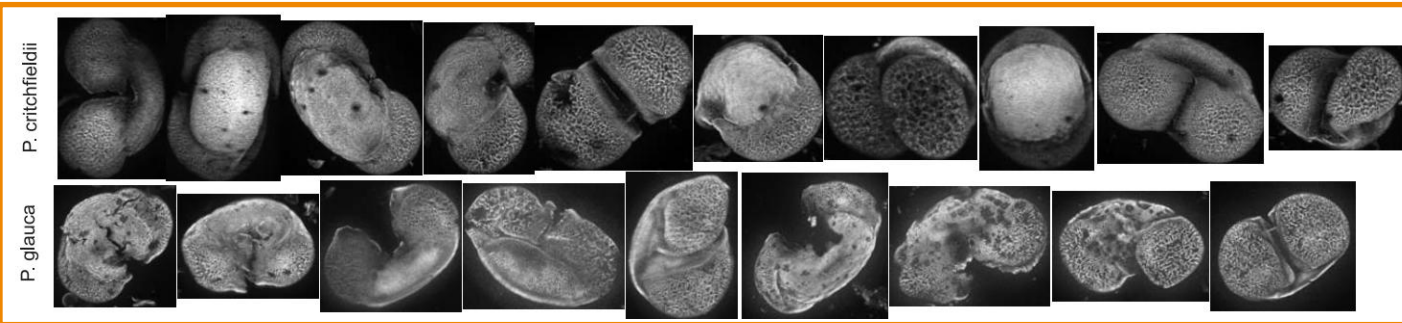


a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$

Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images

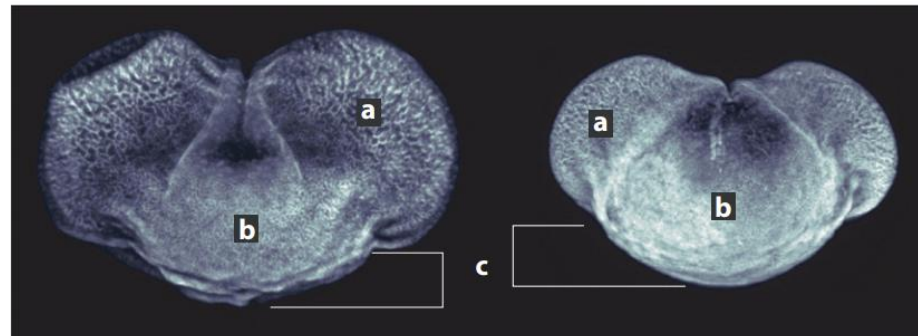


a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$

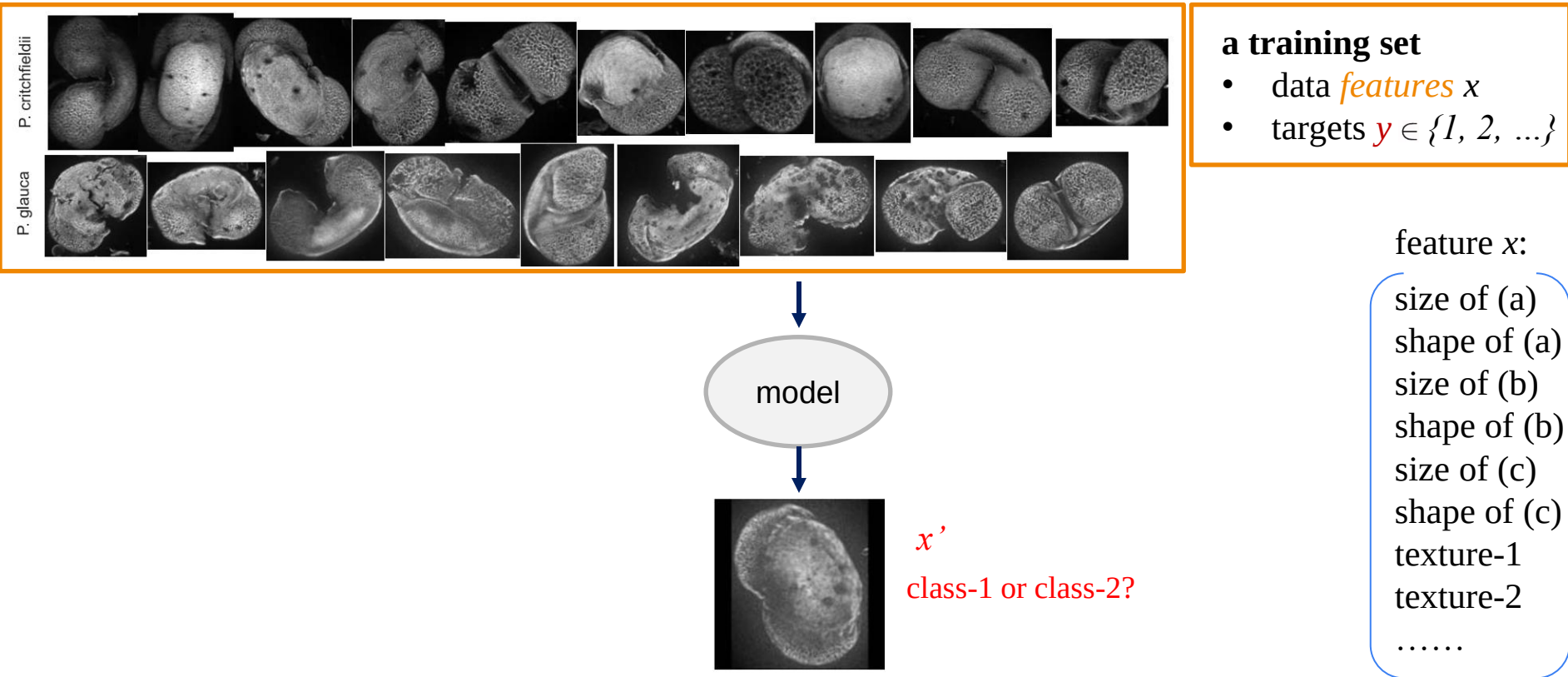
feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....



Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images



- Train a model (i.e., teach a machine) to recognize the species of a test image x'

Nearest Neighbor Classifier

Find the nearest neighbor to x' in the training dataset

a training set

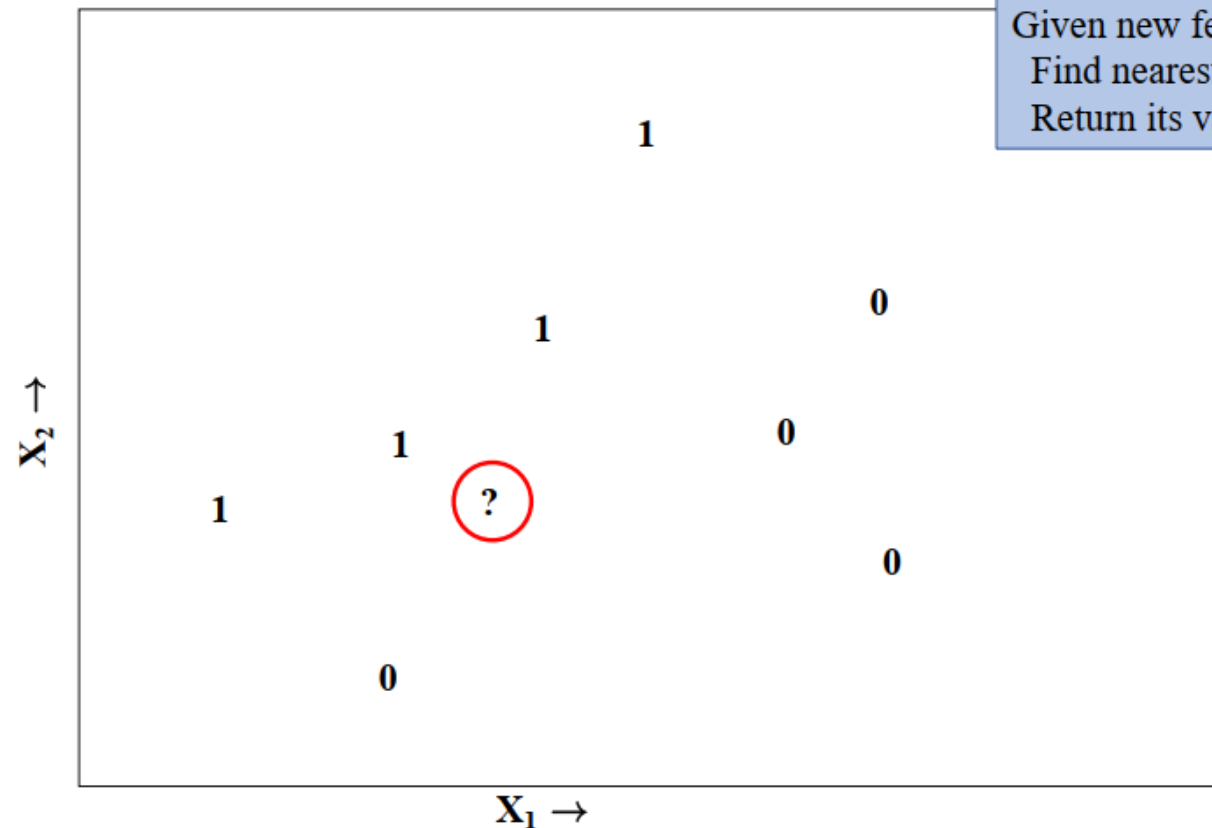
- data *features* x
- targets $y \in \{1, 2, \dots\}$

“Predictor”:

Given new features:
Find nearest example
Return its value

feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....



Nearest Neighbor Classifier

Find the nearest neighbor to x' in the training dataset

1. rank training images according to their Euclidean distance to x'
2. select the training example that has the smallest distance to x'
3. return the corresponding class label as the prediction

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$

“Predictor”:

Given new features:
Find nearest example
Return its value

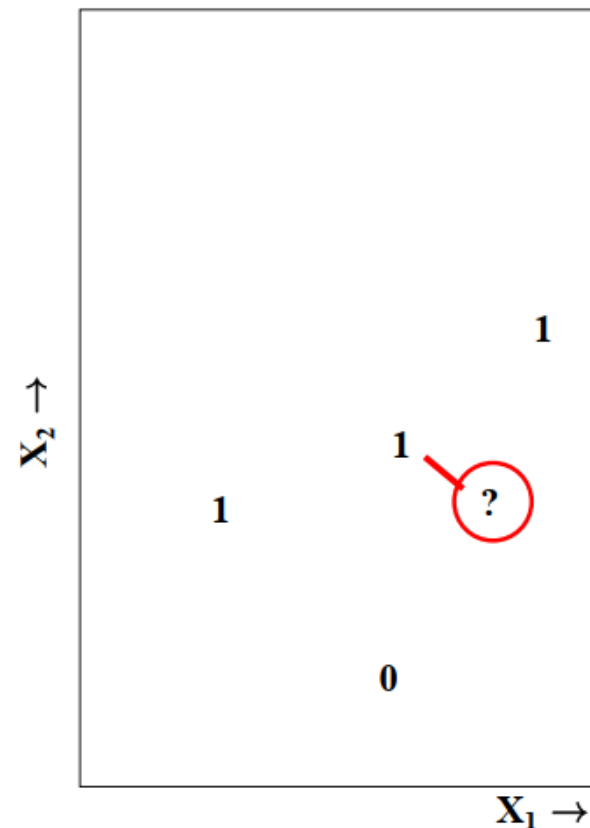
feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

“Closest” training x ?

Typically Euclidean distance:

$$d(x, x') = \sqrt{\sum_i (x_j - x'_j)^2}$$



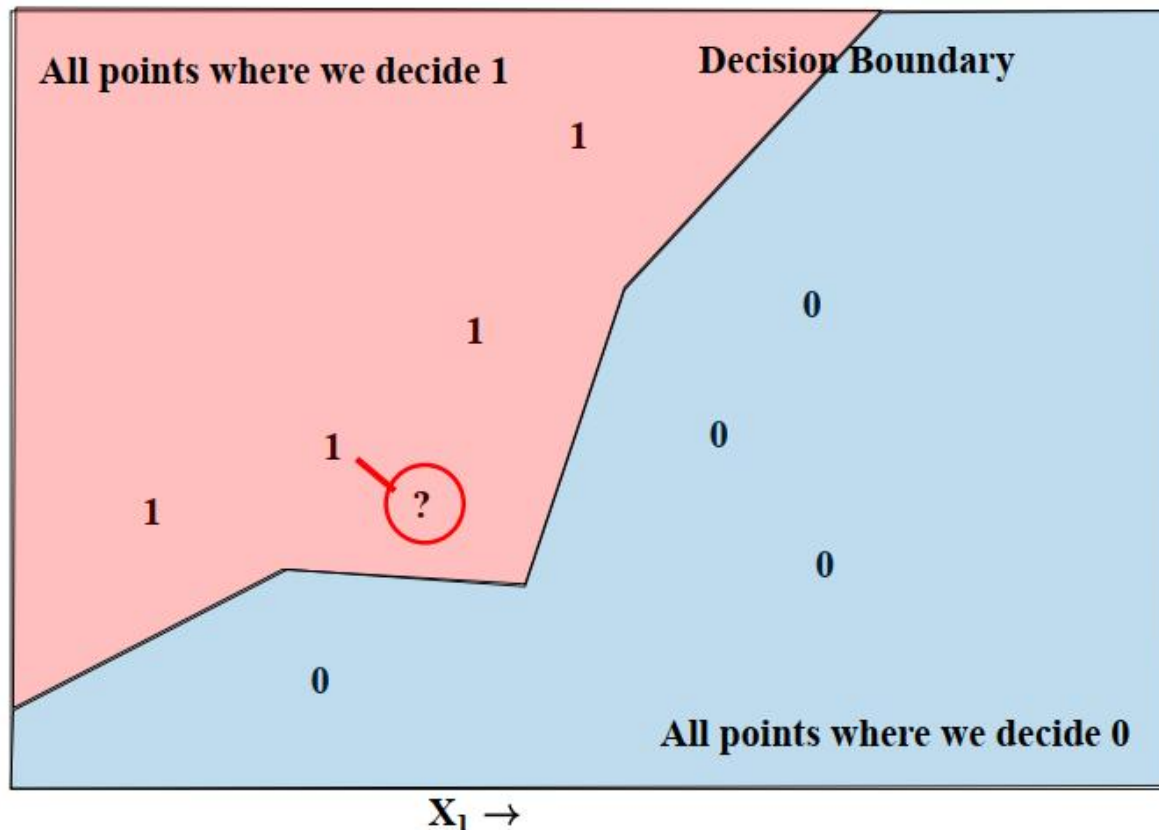
Nearest Neighbor Classifier

Find the nearest neighbor to x' in the training dataset

1. rank training images according to their Euclidean distance to x'
2. select the training example that has the smallest distance to x'
3. return the corresponding class label as the prediction

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

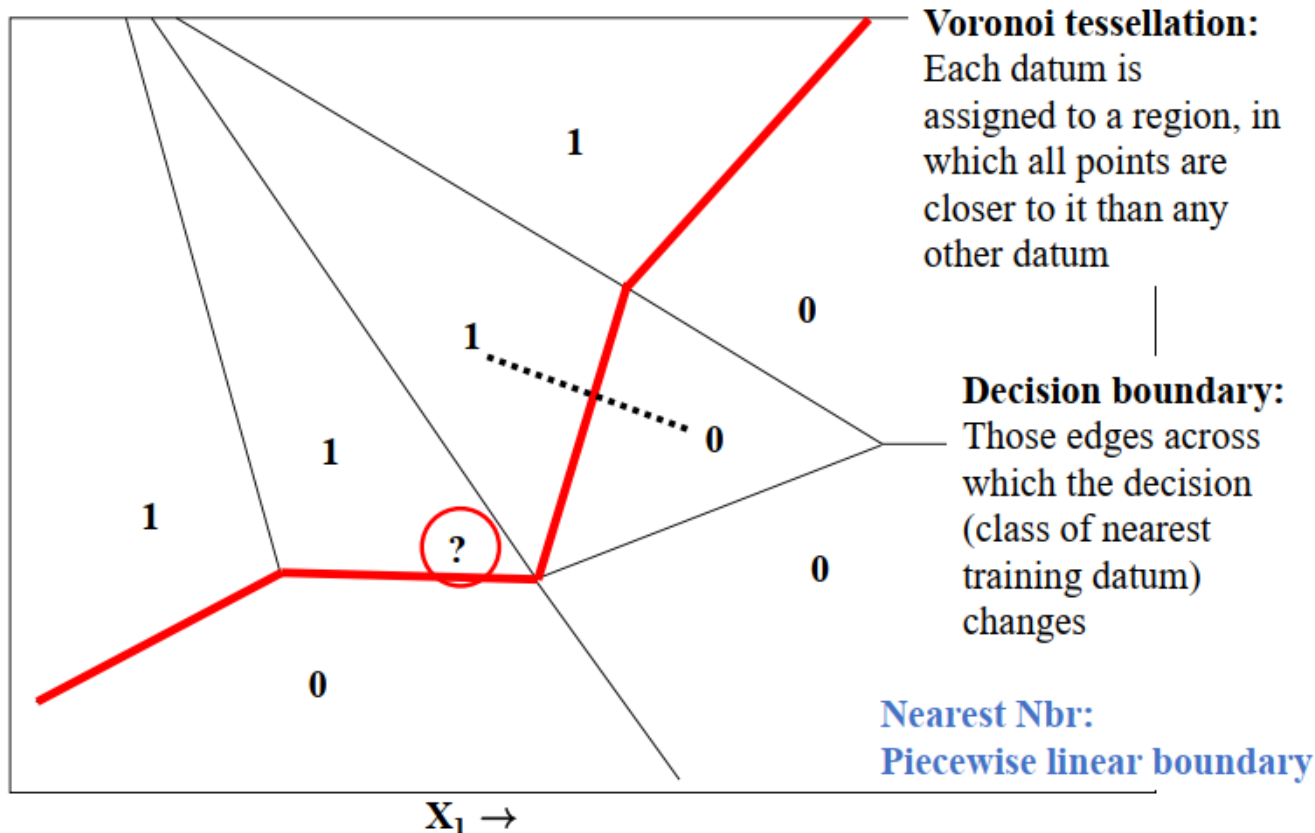
Nearest Neighbor Classifier

Find the nearest neighbor to x' in the training dataset

1. rank training images according to their Euclidean distance to x'
2. select the training example that has the smallest distance to x'
3. return the corresponding class label as the prediction

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

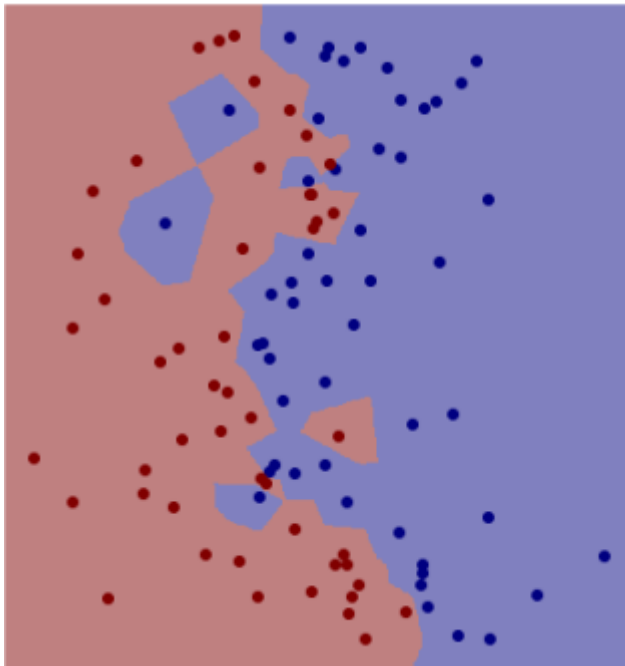
Nearest Neighbor (KNN) model

Find the nearest neighbor to x' in the training dataset

1. rank training images according to their Euclidean distance to x'
2. select the training example that has the smallest distance to x'
3. return the corresponding class label as the prediction

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

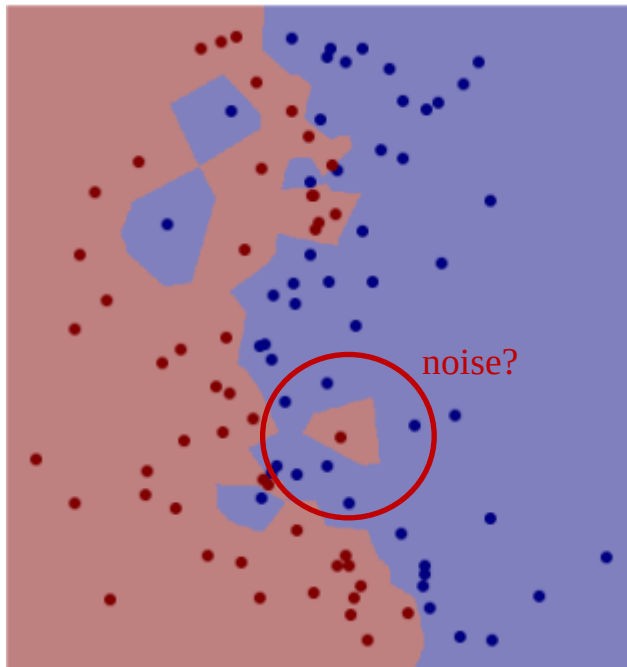
K -Nearest Neighbor (KNN) model

Find the nearest neighbor to x' in the training dataset

1. rank training images according to their Euclidean distance to x'
2. select the training example that has the smallest distance to x'
3. return the corresponding class label as the prediction

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



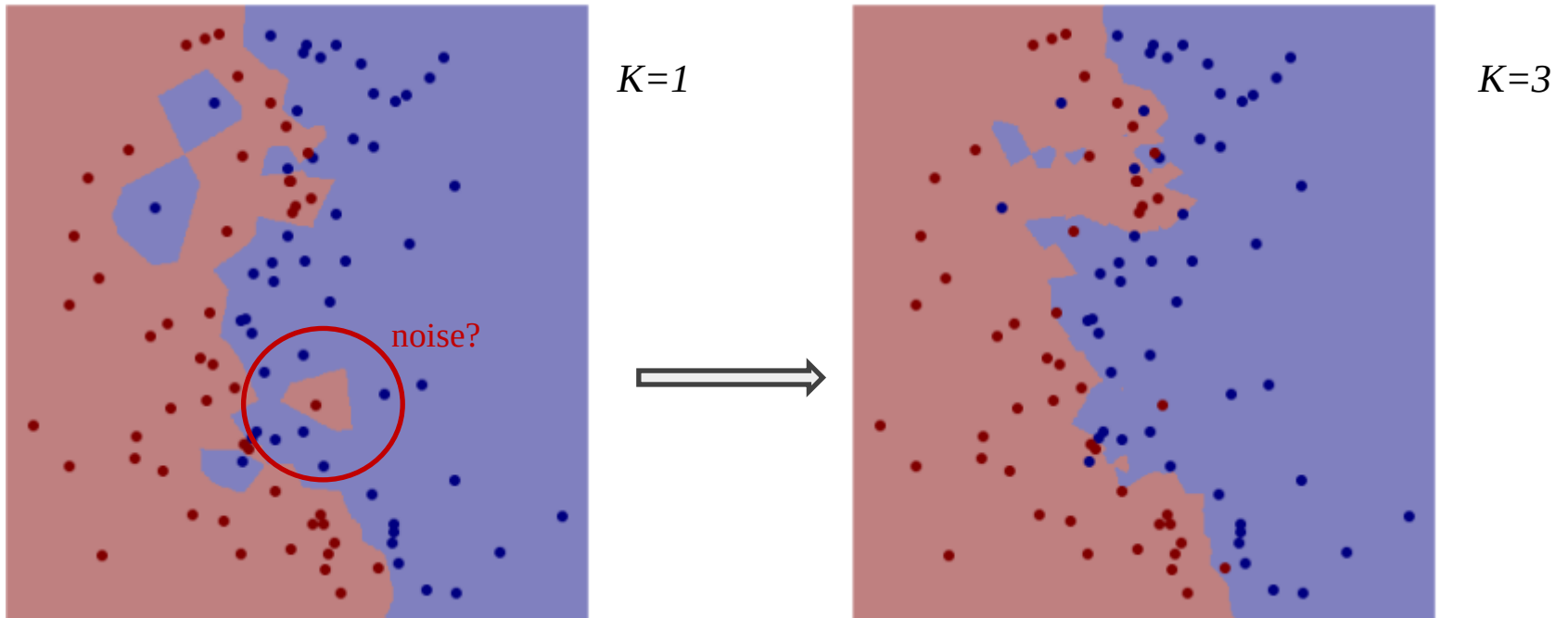
feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

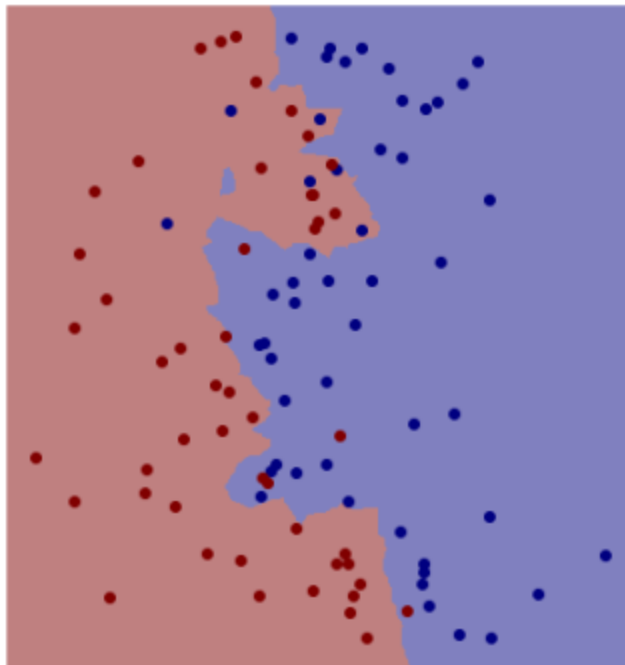
1. rank training images according to their Euclidean distance to x'
2. select K training examples that have the smallest distance to x'
3. vote for the predicted label among these K examples



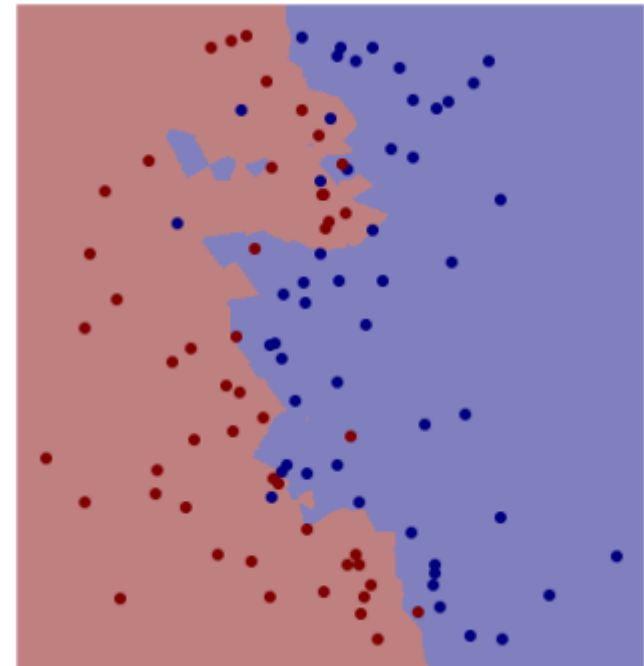
K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to x'
2. select K training examples that have the smallest distance to x'
3. vote for the predicted label among these K examples



$K=5$

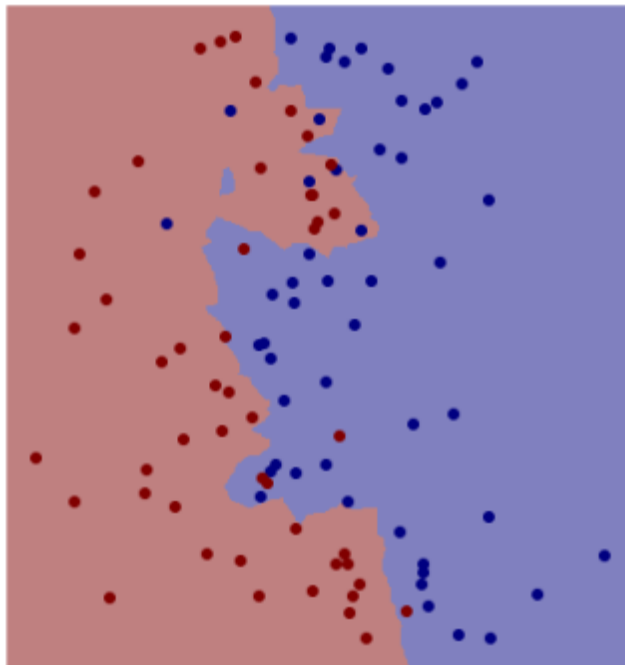


$K=3$

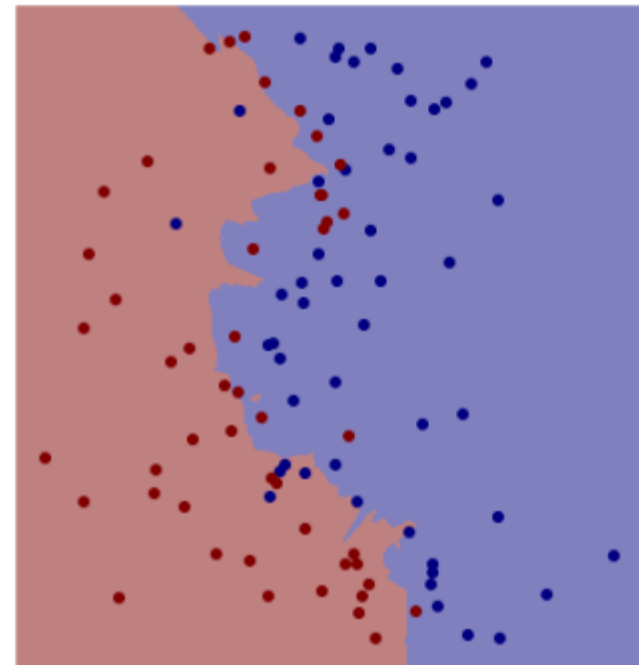
K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to x'
2. select K training examples that have the smallest distance to x'
3. vote for the predicted label among these K examples



$K=5$



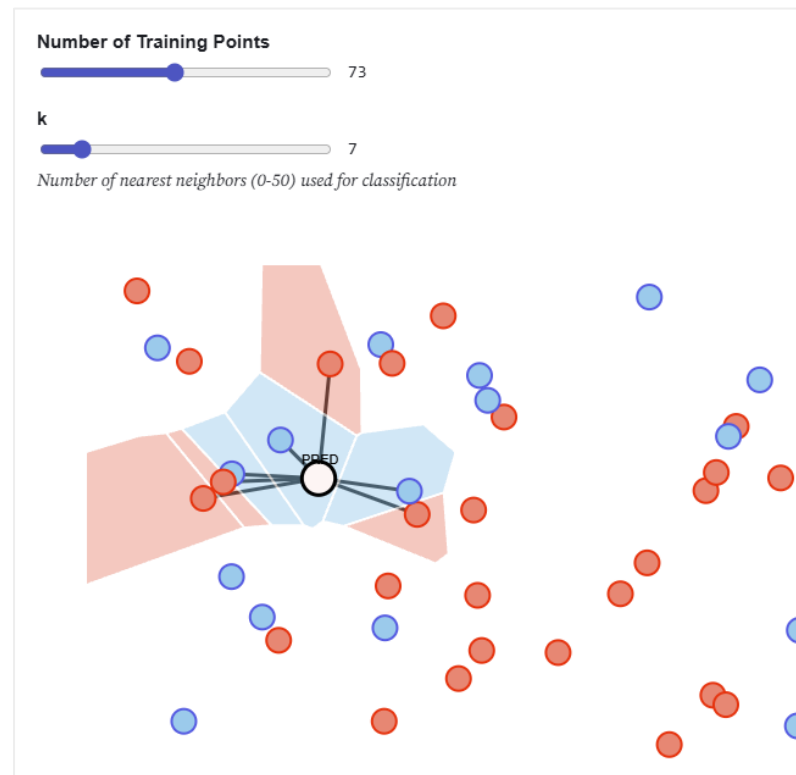
$K=20$

K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to x'
2. select K training examples that have the smallest distance to x'
3. vote for the predicted label among these K examples

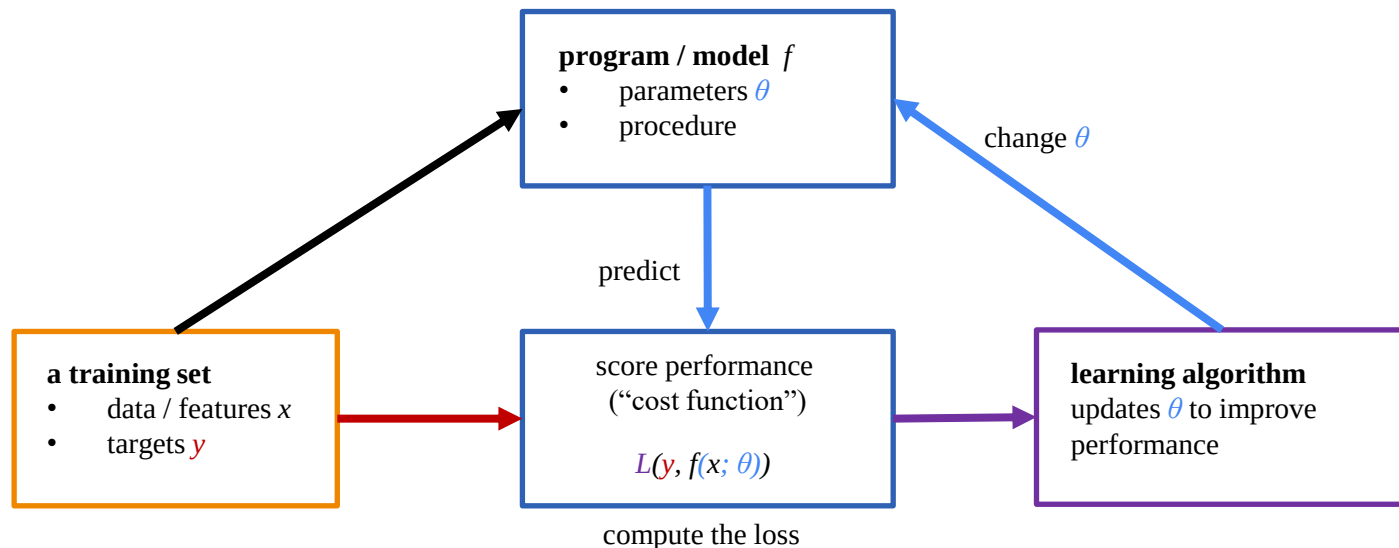
Live demo: <https://observablehq.com/@jwilber/live-k-nearest-neighbors-classification>



K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

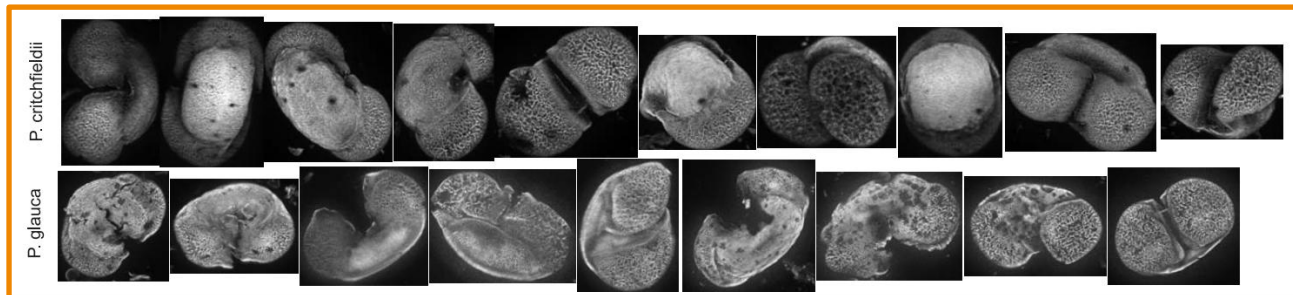
1. rank training images according to their Euclidean distance to x'
 2. select K training examples that have the smallest distance to x'
 3. vote for the predicted label among these K examples
- Given a training set of labeled data
 - Design a model with some parameters
 - Formulate your task as a loss function (e.g., classification errors)
 - Update the parameters to reduce the loss function over your training data



K -Nearest Neighbor (KNN) model

Find the K nearest neighbor to x' in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to x'
2. select K training examples that have the smallest distance to x'
3. vote for the predicted label among these K examples

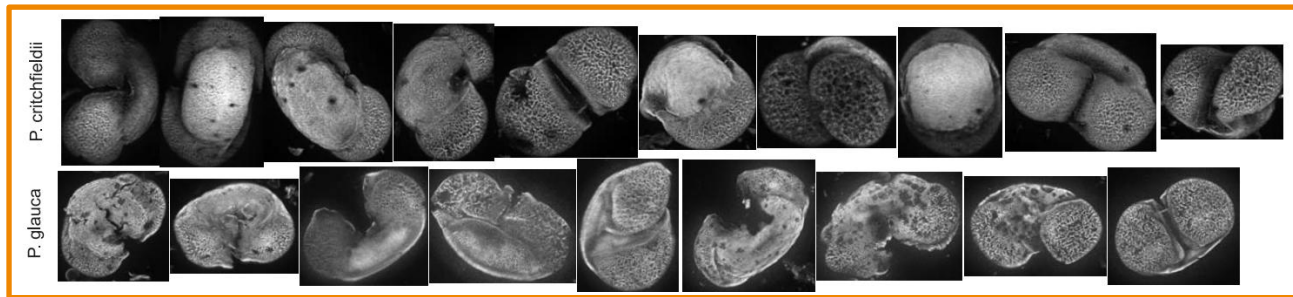


a training set

- data *features* x
- targets y

An extension: Nearest Class Mean

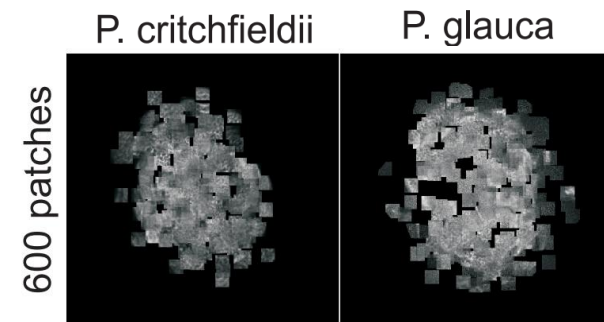
- To reduce storage, use per-class means instead of all training data



a training set

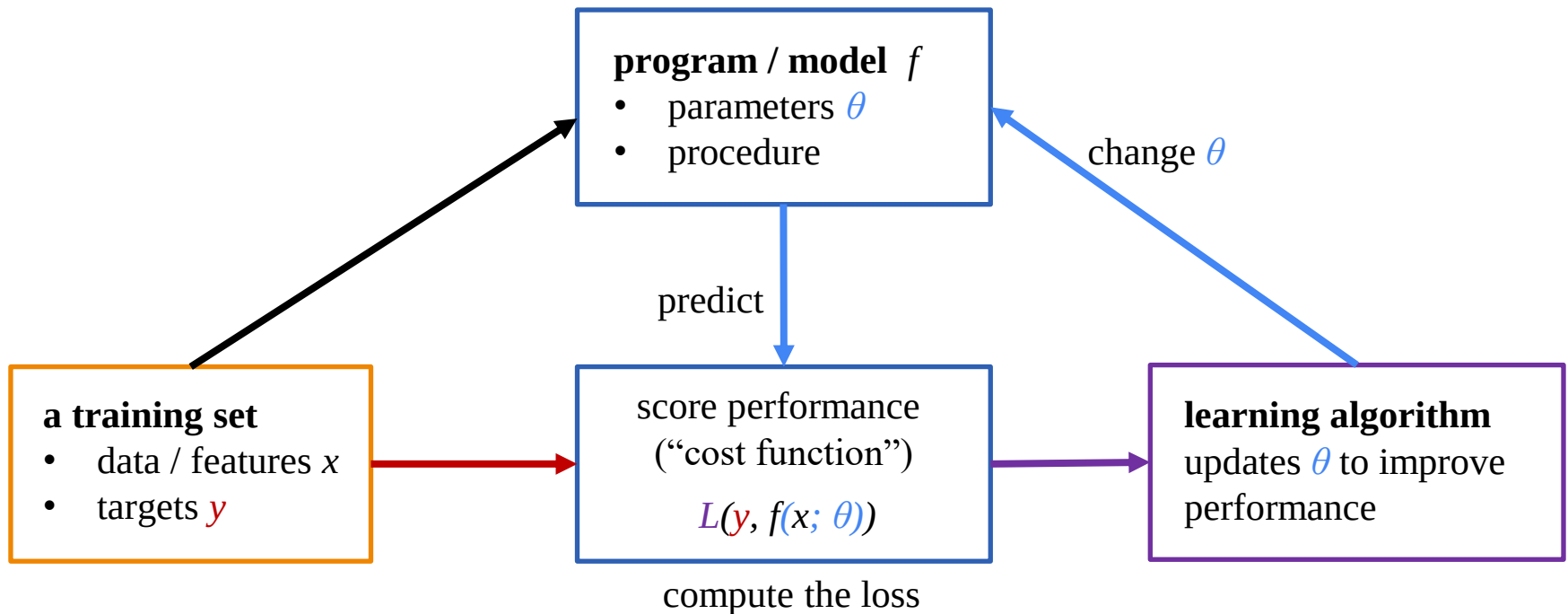
- data *features* x
- targets y

model = {per-class
mean features}



An Overall Pipeline of Machine Learning

- Given a training set of labeled data
- Design a model with some parameters
- Formulate your task as a loss function (e.g., classification errors)
- Update the parameters to reduce the loss function over your training data



Outline

- Data and labels
- Features
- Metrics
- Training, validation, and test sets
- Machine-learning pipeline
- Example: Nearest Neighbor

Hands-on exercise

<https://github.com/aimerykong/OIST-mini-course-CVML>

Today's coding exercise:

- Practice dataset preparation, training/validation splitting
 - Practice image classification using nearest neighbor with raw pixels as features
-
- Download the zip file and upload to your gdrive
 - Download and upload the jupyter notebook to colab

The screenshot displays the GitHub repository page for 'aimerykong / OIST-mini-course-CVML'. The repository is public and has 30 commits. The file structure is as follows:

- dataset
- misc
- slides
- README.md
- code001_part-A_setup.ipynb
- code001_part-B_setup.ipynb
- code001_setup_readme.pdf
- code002_data_prep_NN.ipynb** (highlighted with a red box)

The sidebar on the right shows a list of files with their last commit messages:

Name	Last commit message
..	
dataset_for_classification.zip (circled in red)	Add files via upload
dataset_for_detection.zip	Add files via upload
readme.md	Create readme.md
test-image.jpg	Add files via upload